

Вопросы АиЛОЦУ

СПАСИ И СОХРАНИ

Ч1. Арифметика

1. Позиционные системы счисления

Позиционная система счисления - система, в которой значение цифры зависит от ее места в числе.

Пример: $19 = 1 * 10 + 9 * 1$

$19 \neq 91$ (не равно)

(В непозиционной системе счисления $19 = 1 + 9 = 91$)

2. Представление целого числа и правильной дроби в системе счисления с заданным основанием

Основание r позиционной системы счисления – максимальное количество различных знаков или символов, используемых для изображения числа в данной системе счисления. Таким образом, основание может быть любым числом, кроме единицы и бесконечности. Последовательность степеней основания

называют базисом позиционной системы счисления, каждая из которых задает количественное значение или «вес» каждого разряда. Позиция цифры в числе

называется разрядом.

В общем случае любое число в системе счисления с основанием r может быть записано в общем виде:

$$A = a_n \cdot r^n + a_{n-1} \cdot r^{n-1} + \dots + a_1 \cdot r^1 + a_0 \cdot r^0 + a_{-1} \cdot r^{-1} + \dots + a_{-(m-1)} \cdot r^{-(m-1)} + a_{-m} \cdot r^{-m}, \quad (1)$$

или

$$A_i = \sum_{i=-m}^n a_i r^i, \quad (2)$$

где любая разрядная цифра $a_i \in \{0, \dots, r-1\}$, а каждая степень r^i в такой записи называется весовым коэффициентом разряда; $n+1$ и m соответственно число разрядов целой и дробной части числа.

Старшинство разрядов и соответствующих им цифр определяется значением показателя i (номером разряда). Запись числа в форме (1) назовем записью числа в развернутой форме. Свернутой формой записи чисел называется запись чисел в виде последовательности его r -ичных цифр, перечисляемых по возрастанию старшинства разрядов слева направо:

$$A = a_n a_{n-1} \dots a_0 \dots a_{-(m-1)} a_{-m}.$$

Например, число 214,13 представляется в развернутом виде в десятичной системе счисления:

$$214,13_{10} = 2 \cdot 10^2 + 1 \cdot 10^1 + 4 \cdot 10^0 + 1 \cdot 10^{-1} + 3 \cdot 10^{-2}.$$

3. Двоичная система счисления

Двоичная система счисления — позиционная система счисления с основанием 2. Благодаря непосредственной реализации в цифровых электронных схемах двоичная система используется практически во всех современных компьютерах и прочих вычислительных электронных устройствах.

Двоичная позиционная система счисления имеет основание 2 и использует для записи числа 2 символа (цифры): 0 и 1. В каждом разряде допустима только одна цифра — либо 0, либо 1.

Примером может служить число 101. Оно аналогично числу 5 в десятичной системе счисления. Для того, чтобы перевести из 2-й в 10-ю необходимо умножить каждую цифру двоичного числа на основание “2”, возведенное в степень, равную разряду. Таким образом:

$$101_2 = 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 4 + 0 + 1 = 5_{10}.$$

4. Восьмеричная система счисления

Восьмеричная система — используются 8 символов: 0, 1, 2, 3, 4, 5, 6, 7 — основание системы счисления равно 8.

Для перевода двоичного представления числа в восьмеричное используют правило соответствия каждой восьмеричной цифре триады двоичных цифр.

Любое восьмеричное число можно представить в виде суммы степеней числа-основания системы 8.

5. Шестнадцатеричная система счисления

Шестнадцатеричная система счисления — это позиционная целочисленная система счисления с основанием 16.

Шестнадцатеричная система — это традиционная система счисления с основанием 16. Алфавит состоит из цифр от 0 до 9 и латинских букв от A до F. Латинские буквы представляют собой десятичные числа от 10 до 15.

Развернутая форма записи числа будет выглядеть следующим образом:

$$A_{16} = a_{n-1} \cdot 16^{n-1} + a_{n-2} \cdot 16^{n-2} + \dots + a_0 \cdot 16^0 + a_{-1} \cdot 16^{-1} + \dots + a_{-m} \cdot 16^{-m}$$

Например:

$$3BD_{16} = 3 \cdot 16^2 + B \cdot 16^1 + D \cdot 16^0 = 3 \cdot 256 + 11 \cdot 16 + 13 \cdot 1 = 768 + 176 + 13 = 957_{10}$$

6. Перевод из десятичной системы счисления

Делим десятичное число на основание системы, в которую хотим перевести и записываем остатки от деления. Запишем полученные остатки в обратном порядке и получим искомое число.

Переведем число 375 в восьмеричную систему:

$$375 / 8 = 46 \text{ (остаток 7)}$$

$$46 / 8 = 5 \text{ (остаток 6)}$$

$$5 / 8 = 0 \text{ (остаток 5)}$$

Записываем остатки и получаем 567

7. Перевод из других систем счисления в десятичную систему

Задача перевода сводится к вычислению многочлена:

$$x_P = p_{n-1}P^{n-1} + p_{n-2}P^{n-2} + \dots + p_1P^1 + p_0P^0 + p_{-1}P^{-1} + \dots + p_{-m}P^{-m} = \\ = q_{s-1}Q^{s-1} + q_{s-2}Q^{s-2} + \dots + q_1Q^1 + q_0Q^0 + q_{-1}Q^{-1} + \dots + q_{-r}Q^{-r} = x_Q$$

где P и Q – основание систем счисления и Q = 10

На примере будет легче понять:

$$101100_2 = 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 = 44_{10}$$

8. Двоично-восьмеричные и двоично-шестнадцатеричные преобразования

$0_8 = 000_2$
 $1_8 = 001_2$
 $2_8 = 010_2$
 $3_8 = 011_2$
 $4_8 = 100_2$
 $5_8 = 101_2$
 $6_8 = 110_2$
 $7_8 = 111_2$

$1675_8 = \underbrace{001}_{1} \underbrace{110}_{6} \underbrace{111}_{7} \underbrace{101}_{5}_2$

$1675_8 = 1110111101_2$

Шестнадцатеричная система счисления	Двоичная система счисления
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

я

Двоичное число делится на части по 3(для восьмеричной) или 4(для шестнадцатеричной) знака начиная справа. Каждая триада/тетрада (группа из 3 или 4 двоичных знаков) переводится в нужную систему счисления и записывается одним символом.

Чтобы перевести число из восьмеричной системы в двоичную, нужно каждую восьмеричную цифру записать ее двоичным кодом:

$$156,2_8 = 001\ 101\ 110,010_2$$

Аналогично для шестнадцатеричной, то же самое, только 4 двоичных знака.

9. Двоично-кодированная схема представления десятичных чисел

В современных ПК широко распространена двоично-десятичная система счисления ввиду легкости перевода в десятичную систему и обратно. Она используется там, где основное внимание уделяется не простоте технического построения ЭВМ, а удобству работы пользователя.

В этой системе счисления все десятичные цифры отдельно кодируются четырьмя двоичными цифрами и в таком виде записываются последовательно друг за другом. Например, десятичное число 9703 в двоично-десятичной системе выглядит так:

Десятичное число	9	7	0	3
Двоично- десятичный код	1001	0111	0000	0011

10. Способы кодирования десятичных цифр

В системе кодирования «2 из 5» одна десятичная цифра кодируется 5 битами, из которых 2 бита установлены в 1, а 3 бита — в 0, что даёт ровно 10 комбинаций. Такая система обеспечивает лучшее обнаружение ошибок, поскольку изменение одного бита всегда даст недопустимую комбинацию; также всегда обнаруживаются однонаправленные изменения (несколько изменений $0 \rightarrow 1$ или $1 \rightarrow 0$). Кодирование «2 из 5» использовалось в компьютерах серий IBM 7070, IBM 7072 и IBM 7074; также применяется в некоторых странах для маркирования почты штрих-кодом.

Уплотненные десятичные числа позволяют разместить 3 десятичные цифры в 10 битах ($2^{10}=1024$ комбинации, что достаточно для 3 десятичных цифр), причем кодирование устроено так, что преобразование между 10-битным кодом и тремя отдельными десятичными цифрами можно осуществить с помощью простой и быстрой логической схемы. Такое кодирование используется в десятичных числах с плавающей запятой, описанных в стандарте IEEE 754-2008.

+двоично-десятичное кодирование

8421, 2421, 8421+3, 7421, 2 из 5?

11. Двоично-десятичное кодирование

Двоично-десятичный код, BCD, 8421-BCD — форма записи рациональных чисел, когда каждый десятичный разряд числа записывается в виде его четырёх битного двоичного кода.

В коде 8421 разрешенные комбинации соответствуют двоичным эквивалентам десятичных цифр с весами разрядов, равными степеням основания 2

Каждая цифра десятичного числа представляется в двоичной форме и изображается соответствующим 4-разрядным двоичным числом (тетрадой)

Код 8421 является взвешенным кодом

Цифры, являющиеся дополнением до 9, не получаются простым инвертированием тетрад

Важно понимать, что двоично-десятичный код:

не является еще одной системой счисления

не то же самое, что обычный двоичный код. Обычный двоичный код представляет в двоичной системе какое-либо десятичное число полностью, а двоично-десятичный код преобразовывает в двоичную систему каждый десятичный разряд по отдельности

Код 8421 является естественным представлением десятичных цифр в двоичной системе и удобен для ввода/вывода в цифровых устройствах десятичных чисел

12. Представление чисел с фиксированной запятой

В ЭВС числа и нечисловая информация представляются совокупностью двоичных разрядов. Совокупность двоичных разрядов, предназначенных для представления(записи) данных, называется разрядной сеткой.

В ЭВС применяют две формы представления чисел:

- с фиксированной запятой(точкой)
- с плавающей запятой(точкой)

В форме с ФЗ запятая, отделяющая дробную часть от целой, фиксируется в определенном месте относительно разрядов числа.

При этом в слове данных сохраняются только два структурных компонента:

- один знаковый разряд
- n разрядов для представления цифр числа

Для кода знака выделяется крайний слева разряд (в знаковом разряде 1 соотв. минусу, 0 – плюсу)

Положения запятой фиксируется либо после младшего разряда (0 разряда), либо перед старшим разрядом (n-1 разрядом). В первом случае числа представлены как целые, во втором – как правильные дроби.

Разрядная сетка для представления целых чисел

$$A_{\min} = 00\dots01, = 1 \quad A_{\max} = 11\dots11, = 2^n - 1 \quad 1 \leq |A| \leq 2^n - 1$$

n	$n-1$	$n-2$	$\dots$	1	0
$\pm$	a_{n-1}	a_{n-2}	$\dots$	a_1	a_0
	2^{n-1}	2^{n-2}	$\dots$	2^1	2^0

Разрядная сетка для представления правильных дробей

$$A_{\min} = ,00\dots01 = 2^{-n} \quad A_{\max} = ,11\dots11 = 1 - 2^{-n} \quad 2^{-n} \leq |A| \leq 1 - 2^{-n}$$

n	$n-1$	$n-2$	$\dots$	1	0
$\pm$	a_{-1}	a_{-2}	$\dots$	$a_{-(n-1)}$	a_{-n}
	2^{-1}	2^{-2}	$\dots$	$2^{-(n-1)}$	2^{-n}

13. Представление чисел с плавающей запятой

Вещественные числа хранятся в памяти компьютера в специальной форме с плавающей точкой (с плавающей запятой).р, причем оба эти числа – целые и могут иметь знак. Такое число представлено в виде произведения $m \cdot 2^p$. В одной ячейке памяти хранится два значения – мантисса m и порядок

Пример: $155.625_{10} = 10011011.101_2$

Степень 2:	...	10	9	8	7	6	5	4	3	2	1	0		-1	-2	-3	-4	-5	...
	...	0	0	0	1	0	0	1	1	0	1	1	•	1	0	1	0	0	...
Сумма:					128			16	8		2	1		0.5		0.125			

Нормализация: $155.625_{10} = 1.55625 \times 10^2$

$$10011011.101_2 = 1.0011011101 \times 2^{111} = m \cdot 2^p$$

m – мантисса p – порядок (экспонента)

Мантиссу нормализуют так, чтобы она удовлетворяла условию $1 \leq m < 2$, то есть ненулевая мантисса всегда должна начинаться с двоичной единицы. В этом случае имеет смысл хранить только дробную часть мантиссы как целое число, а целую часть не хранить и считать всегда равной единице.>, то есть ненулевая мантисса всегда должна начинаться с двоичной единицы. В этом случае имеет смысл хранить только дробную часть мантиссы как целое число, а целую часть не хранить и считать всегда равной единице.

Стандарт представления

Формат представления чисел с плавающей точкой определяется международным стандартом IEEE 754 (IEC 60559), который описывает числа: - одинарной точности, 32-bit; - двойной точности, 64-bit; - двойной расширенной точности, обычно 80-bit. Этот стандарт широко используется в программных и аппаратных реализациях арифметических действий с числами с плавающей точкой.

Двойная точность (double precision), 64-bit, double

Структура чисел с плавающей точкой **двойной точности** аналогична, но занимает 8 байт = 64 бита:

8 байт = 64 бита = 1 (знак) + 11 (порядок) + 52 (мантисса)

Число **двойной расширенной точности** (double-extended precision) в соответствии со стандартом занимает ≥ 79 бит, но обычно ограничиваются 80 битами.

Исключения

Формат хранения чисел с плавающей точкой приводит к появлению нескольких исключительных ситуаций:

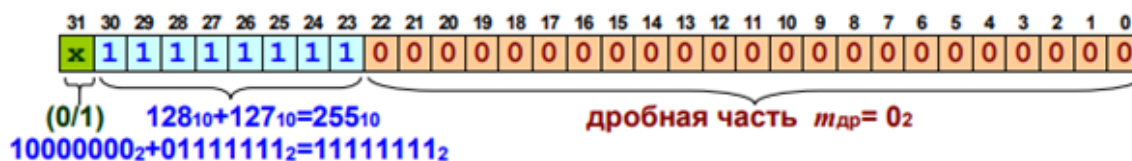
- 1) Нуль может быть закодирован двумя способами: +0 и -0. Однако эти значения при вычислениях не различаются.



- 2) Денормализованные числа: у денормализованных чисел мантисса удовлетворяет условию $0.1 \leq m < 1$. Такие числа разбивают минимальный разряд нормализованного числа на некоторое подмножество. >.



- 3) «Бесконечность»: Бесконечным считается число с максимальным порядком (все единицы) и нулевой дробной частью мантиссы (все нули). Различаются $+\infty$ и $-\infty$.



- 4) «Не число»: Не считаются числами коды с максимальным порядком (все единицы) и ненулевой дробной частью мантиссы (любая ненулевая комбинация 0 и 1).



14. Погрешности представления чисел

Все числа целого типа переводятся из одной системы счисления в другую точно (без погрешностей). Операции сложения, вычитания и умножения чисел целого типа также выполняются точно. Результат деления чисел целого типа дает целое частное и целый остаток. Правильные дроби переводятся из одной системы счисления в другую не точно – с погрешностью.

Некоторая величина в одной системе счисления может иметь конечное значение, а в другой системе счисления становится бесконечной величиной.

Например, дробь $1/10$ имеет конечное десятичное представление, но, будучи переведена в двоичную систему счисления, становится бесконечной дробью $0,0001100110011...$

Числа с фиксированной запятой

- Для чисел с ФЗ погрешность представления имеет место только в случае правильных дробей.
- Абсолютная погрешность перевода десятичной дроби в систему с основанием r при использовании усечения (младшие цифры справа отбрасываются) определяется следующим выражением

$$\begin{aligned}\Delta[A] &= a_{-(n+1)}r^{-(n+1)} + a_{-(n+2)}r^{-(n+2)} + \dots \\ &= \sum_{i=-(n+1)}^{-\infty} a_i r^i.\end{aligned}$$

- Для двоичной системы счисления $r = 2$, и максимальное значение этой погрешности получается при $a_i = 1$

$$\Delta[A]_{max} = \sum_{i=-(n+1)}^{-\infty} 1 \cdot 2^i = 2^{-n} \sum_{i=-1}^{-\infty} 2^i = 2^{-n},$$

т. е. максимальная абсолютная ошибка равна значению младшего разряда.

- При использовании округления максимальная абсолютная ошибка не превышает половины значения младшего разряда

$$\Delta[A]_{max} = 2^{-(n+1)} = 0,5 \cdot 2^{-n}.$$

- Для представления чисел в форме с ФЗ абсолютное значение машинного изображения правильной дроби находится в диапазоне

$$2^{-n} \leq |A|_{\Phi} \leq 1 - 2^{-n}.$$

- Следовательно, относительная погрешность представления для максимального значения числа равна

$$\delta[A]_{\Phi min} = \frac{\Delta[A]}{|A|_{\Phi max}} = \frac{0,5 \cdot 2^{-n}}{1 - 2^{-n}}.$$

- В современных ЭВС, как правило, $n = 16 \dots 64$, поэтому $1 \gg 2^{-n}$, откуда

$$\delta[A]_{\Phi min} = 0,5 \cdot 2^{-n}.$$

Числа с плавающей запятой

- Все числа с ПЗ представляют некоторые данные приближенно.
- Для представления чисел в форме с ПЗ абсолютное значение мантиссы находится в диапазоне

$$0,5 \leq |M| < 1 - 2^{-n},$$

поэтому ее погрешности определяются так же, как и для чисел с фиксированной запятой.

15. Кодирование двоичных чисел со знаком: прямой код

Все описано в таблице 17. Stonks! Woman moment

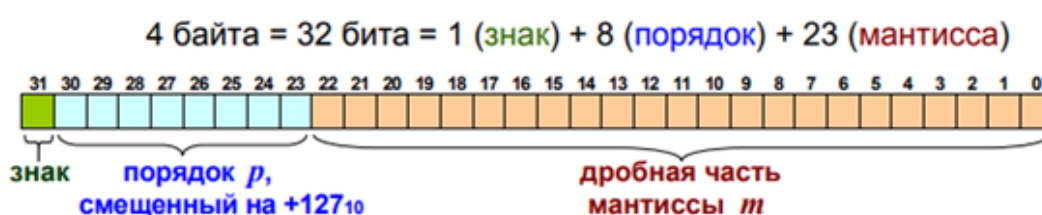
16. Кодирование двоичных чисел со знаком: дополнительный код

Все описано в таблице 17. Woman moment

17. Кодирование двоичных чисел со знаком: обратный

Одинарная точность (single precision), 32-bit, float

Структура формата хранения вещественных чисел с плавающей точкой одинарной точности, занимающих в памяти ячейку размером 4 байта = 32 бита:



Для хранения двоичного кода порядка p используется 8 бит. Чтобы значение p могло быть отрицательным к нему прибавляют смещение – число $127_{10} = 01111111_2$. Таким образом можно хранить целые десятичные значения p от -127 до $+128$.

КОД Пример:

$$\pm 7,25_{10} = \pm 111,01_2$$

Положительные числа	Отрицательные числа
---------------------	---------------------

Обратный = Дополнительный = Прямой	Обратный: инверсия Дополнительный: Инверсия прямого кода, +1 в крайний справа разряд Если число дробное, то +1 в крайний справа разряд дробной части
---------------------------------------	--

	+7,25	-7,25
Прямой	0.111,01	1.111,01
Обратный	0.111,01	1.000,10 (инверсия)
Дополнительный	0.111,01	1.000,11 (инверсия, +1)

18. Сложение и вычитание двоичных чисел в форме с фиксированной запятой со знаком в прямых кодах. Особенности выполнения операций над числами без знака

Таблица сложения	Таблица умножения	Таблица вычитания
0 + 0 = 0	0 * 0 = 0	0 - 0 = 0
0 + 1 = 1	0 * 1 = 0	1 - 0 = 1
1 + 0 = 1	1 * 0 = 0	1 - 1 = 0
1 + 1 = 10	1 * 1 = 1	10 - 1 = 1

Это обычный двоичный код, рассмотренный в разделе двоичной системы счисления. Если двоичное число является положительным, то бит знака равен 0, если двоичное число отрицательное, то бит знака равен 1. Цифровые разряды прямого кода содержат модуль представляемого числа, что обеспечивает наглядность представления чисел в прямом коде (ПК).

Рассмотрим однобайтовое представление двоичного числа. Пусть это будет $28_{(10)}$. В двоичном формате – $0011100_{(2)}$ (при однобайтовом формате под величину числа отведено 7 разрядов). Двоичное число со знаком будет выглядеть так, как показано на рис. 2.1.

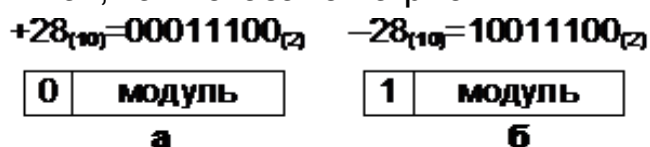


Рис. 2.1. Формат двоичного числа со знаком в прямом коде:
а – положительное число, б – отрицательное

Сложение в прямом коде чисел, имеющих одинаковые знаки, достаточно просто: числа складываются, и сумме присваивается знак слагаемых. Значительно более сложным является алгебраическое сложение в прямом коде чисел с разными знаками. В этом случае приходится определять большее по модулю число, производить вычитание модулей и присваивать разности знак большего по модулю числа. Такую операцию значительно проще выполнять, используя обратный и дополнительный коды.

19. Сложение и вычитание двоичных чисел в форме с фиксированной запятой со знаком в дополнительных кодах

При выполнении арифметических операций в современных ЭВМ используется представление положительных чисел в прямом коде (ПК), а отрицательных – в обратном (ОК) или в дополнительном (ДК) кодах. Это можно проиллюстрировать схемой на рис. 2.4.

ПК	0	Мод. дв. числа	1	Мод. дв. числа
ОК	0	Мод. дв. числа	1	Мод. дв. числа
ДК	0	Мод. дв. числа	1	ОК+1 _{МЗР}
	а		б	

Рис. 2.4. Представление чисел в ЭВМ:
а – положительное число; б – отрицательное число

Общее правило. При алгебраическом сложении двух двоичных чисел, представленных обратным (или дополнительным) кодом, производится арифметическое суммирование этих кодов, включая разряды знаков. При возникновении переноса из разряда знака единица переноса прибавляется к МЗР суммы кодов при использовании ОК и отбрасывается при использовании ДК. В результате получается алгебраическая сумма в обратном (или дополнительном) коде.

Рассмотрим подробнее алгебраическое сложение для случая представления отрицательных чисел в ДК.

При алгебраическом сложении чисел со знаком результатом также является число со знаком. Суммирование происходит по всем разрядам, включая знаковые, которые при этом рассматриваются как старшие. При переносе из старшего разряда единица переноса отбрасывается и возможны два варианта результата:

- знаковый разряд равен нулю: результат – положительное число в ПК;
- знаковый разряд равен единице: результат – отрицательное число в ДК.

Для определения абсолютного значения результата его необходимо инвертировать, затем прибавить единицу.

Пример.

Вычислить алгебраическую сумму 58-23.

+

$$\begin{array}{rcl}
 58_{(10)} & \rightarrow & 0011\ 1010_{(2)} \quad - \text{ПК} \\
 -28_{(10)} & \rightarrow & 1001\ 0111_{(2)} \quad - \text{ПК} \\
 & & 1110\ 1001_{(2)} \quad - \text{ДК}
 \end{array}$$

Число отрицательное - необходимо перевести в ДК (быстрый перевод)

$$\begin{array}{r}
 + 0011\ 1010 \\
 + 1110\ 1001 \\
 \hline
 1 \leftarrow 0010\ 0011_{(2)} \quad (\text{ПК}) = 35_{(10)}
 \end{array}$$

Перенос из знакового разряда отбрасываем.
Число является положительным в ПК.
перенос

На картинке опечатка вместо -28 должно быть -23!

Пример.

Вычислить алгебраическую сумму 26-34.

$$\begin{array}{rcl}
 26_{(10)} & \rightarrow & 0001\ 1010_{(2)} \quad - \text{ПК} \\
 -34_{(10)} & \rightarrow & 1010\ 0010_{(2)} \quad - \text{ПК} \\
 & & 1101\ 1110_{(2)} \quad - \text{ДК}
 \end{array}$$

Быстрый перевод

$$\begin{array}{r}
 + 0001\ 1010 \\
 + 1101\ 1110 \\
 \hline
 0 \leftarrow 1111\ 1000_{(2)}
 \end{array}$$

Перенос из знакового разряда отсутствует. Число отрицательное в ДК, так как знаковый разряд равен 1.
перенос

Вычислим результат, преобразовав его из ДК в ПК :

$$\begin{array}{r}
 1111\ 1000_{(2)} \quad (\text{ДК}) \\
 + 1000\ 0111 \quad \text{Инверсия всех разрядов, кроме знакового} \\
 + \quad \quad \quad 1 \\
 \hline
 1000\ 1000_{(2)} \quad (\text{ПК}) = -8_{(10)}
 \end{array}$$

Пример.

Вычислить алгебраическую сумму -5-1.

$$\begin{array}{rcl}
 -5_{(10)} & \rightarrow & 1101\ (\text{ПК}) \rightarrow 1011\ (\text{ДК}) \\
 -1_{(10)} & \rightarrow & 1001\ (\text{ПК}) \rightarrow 1111\ (\text{ДК})
 \end{array}$$

$$\begin{array}{r}
 + 1011 \\
 + 1111 \\
 \hline
 1 \leftarrow 1010_{(2)}
 \end{array}$$

Перенос из знакового разряда отбросить.
Число отрицательное в ДК, так как знаковый разряд равен 1.
перенос

Запишем результат, преобразовав его из ДК в ПК :

$$\begin{array}{r}
 1010_{(2)} \quad (\text{ДК}) \\
 + 1101 \quad \text{Инверсия значащих разрядов (кроме знакового)} \\
 + \quad \quad \quad 1 \\
 \hline
 1110_{(2)} \quad (\text{ПК}) = -6_{(10)}
 \end{array}$$

20. Сложение и вычитание двоичных чисел в форме с фиксированной запятой со знаком в обратных

кодах

Теорема. Сумма обратных кодов чисел есть обратный код суммы чисел.

Доказательство. Предположим, что числа представлены в форме с фиксированной запятой, стоящей после младшего разряда, т. е. слагаемые являются целыми числами. Рассмотрим возможные случаи.

1. $A > 0, B > 0, A + B < 2^n$.

Так как $[A]_{\text{обр}} = A, [B]_{\text{обр}} = B$, то $[A]_{\text{обр}} + [B]_{\text{обр}} = \underbrace{A + B}_{\text{положительное число}} = [A + B]_{\text{обр}}$.

2. $A < 0, B > 0, |A| > B$.

Так как $[A]_{\text{обр}} = 2^{n+1} - 1 + A, [B]_{\text{обр}} = B$, то

$$[A]_{\text{обр}} + [B]_{\text{обр}} = 2^{n+1} - 1 + \underbrace{(A + B)}_{\text{отрицательное число}} = [A + B]_{\text{обр}}$$

3. $A < 0, B > 0, |A| < B$.

Так, как и в предыдущем случае, здесь

$$[A]_{\text{обр}} = 2^{n+1} - 1 + A, [B]_{\text{обр}} = B, [A]_{\text{обр}} + [B]_{\text{обр}} = 2^{n+1} - 1 + (A + B).$$

Однако в этом случае $(A + B)$ число положительное, и поэтому

$$2^{n+1} - 1 + (A + B) > 2^{n+1}.$$

Так как значение этой суммы больше 2^{n+1} , то появляется единица переноса из знакового разряда, что равносильно изъятию из суммы 2^{n+1} единиц. Кроме того, сумма получается уменьшенной на единицу. Если отбросить единицу переноса из знакового разряда и добавить ее в младший разряд суммы, то получим

$$[A]_{\text{обр}} + [B]_{\text{обр}} = 2^{n+1} - 1 + \underbrace{(A + B)}_{\text{положительное число}} = A + B = [A + B]_{\text{обр}}$$

единица переноса из
знакового разряда,
которая циклически
прибавляется к
младшему разряду

В этом случае получается отрицательный результат и также появляется единица переноса из знакового разряда, которая добавляется в младший разряд суммы.

При сложении чисел с использованием обратного кода выполняется арифметическое сложение кодов слагаемых, включая их знаковые разряды. При возникновении переноса из знакового разряда суммы единица переноса прибавляется к младшему разряду суммы. Такой перенос называется круговым, или циклическим. При сложении код знака результата получается автоматически.

Пример 10. Сложить два числа в обратных кодах: $A = +1001$, $B = -101$ ($n = 4$).

Решение. $[A]_{\text{обр}} = 0.1001$; $[B]_{\text{обр}} = 1.1010$

$$\begin{array}{rcl}
 0.1001 & [A]_{\text{обр}} & \\
 + 1.1010 & [B]_{\text{обр}} & \\
 \hline
 \boxed{1}0.0011 & & \\
 + \xrightarrow{\quad} \boxed{1} & \text{циклический перенос} & \\
 \hline
 0.0100 & [A + B]_{\text{обр}} = [A + B]_{\text{пр}} &
 \end{array}$$

Ответ. $A + B = +100$.

Таким образом, сложение с использованием обратных кодов выполняется, как и в случае дополнительных кодов, по обычным правилам сложения двоичных чисел. При этом знаковый разряд рассматривается наравне со значащими разрядами. Однако в отличие от сложения с использованием дополнительного кода единица переноса из знакового разряда не отбрасывается, а прибавляется к младшему разряду суммы.

Следует помнить, что при сложении двух одинаковых по абсолютной величине чисел с разными знаками в случае применения обратного кода получается отрицательный ноль $1.11...11$. Однако отрицательный ноль в ЭВС, как правило, автоматически преобразуется к положительному, т. е. к виду $0.00...00$.

Сложение правильных дробей ничем не отличается от сложения целых чисел.

Операция вычитания чисел в ЭВС сводится к операции сложения с использованием дополнительного и обратного кодов.

21. Переполнение при сложении чисел с фиксированной запятой. Модифицированные коды

При сложении чисел одинакового знака, представленных в форме с фиксированной запятой, может возникнуть переполнение разрядной сетки. Это когда получившаяся сумма имеет больше разрядов, чем предусмотрено. Например: $13 + 5 = 48 - 2$ (!)

0	1	1	0	1
---	---	---	---	---

0	0	1	0	1
1	0	0	1	0

Где красным выделен знаковый разряд.

Как видим, главным показателем переполнения является разность знаков слагаемых и суммы. Решается эта проблема разными методами, но сперва необходимо заметить переполнение. И самым используемым из способов является использование модифицированных кодов. В них знаковая часть представляется 2 одинаковыми битами, 00 для положительного и 11 для отрицательного числа. Если после операции оба бита равны – значит переполнения не было во время операции. Если же получаем 01 или 10 – операция привела к переполнению и с этим нужно что-то делать(остановка операции/выведение специального сигнала):

0	0	1	1	0	1
0	0	0	1	0	1
0	1	0	0	1	0

22. Умножение чисел с фиксированной запятой: общая схема целочисленного умножения

Традиционная схема умножения похожа на известную из школьного курса процедуру записи «в столбик». В общем случае вычисление произведения двух n -разрядных двоичных чисел без знака множимого и множителя

сводится к формированию частичных произведений по одному на каждую цифру множителя с последующим суммированием полученных ЧП. Перед суммированием каждое частичное произведение должно быть сдвинуто на один разряд относительно предыдущего согласно весу цифры.

	a_3	a_2	a_1	a_0	A				
	$\times b_3$	b_2	b_1	b_0	B				
		$w_{03}w_{02}w_{01}w_{00}$			$W_0 = Ab_0 \times 2^0$				
+		$w_{13}w_{12}w_{11}w_{10}$			$W_1 = Ab_1 \times 2^1$				
		$w_{23}w_{22}w_{21}w_{20}$			$W_2 = Ab_2 \times 2^2$				
		$w_{33}w_{32}w_{31}w_{30}$			$W_3 = Ab_3 \times 2^3$				
	p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0	P

Рисунок 6 – Общая схема умножения для $n = 4$

Алгоритм умножения предполагает последовательное выполнение двух операций – сложения и сдвига. Суммирование ЧП обычно производится по мере их получения (это сокращает аппаратные затраты). Процесс получения произведения включает умножение множимого A на каждый разряд b_i множителя B. Получаемые при этом частичные произведения последовательно складываются, образуя *суммы частичных произведений*.

Цикл умножения – это процесс умножения n -разрядного двоичного множимого A на n -разрядный двоичный множитель B, состоящий из повторяющихся n раз последовательности умножения A на каждую очередную цифру B.

Умножение чисел со знаком может выполняться как в прямом, так и в дополнительном коде. При умножении чисел в прямом коде перемножаются множители сомножителей A и B, а знак произведения определяется путем сложения по модулю двух знаковых разрядов сомножителей и приписывается произведению после завершения умножения.

Выполнение операции умножения в дополнительном коде осуществляется в соответствии с *алгоритмом Робертсона* (аналогичен алгоритму для чисел без знака по четвертому методу), но учитывает две принципиальные особенности, обусловленные дополнительным кодом.

23. Методы умножения двоичных чисел без знака

*Процесс умножения двоичных чисел полностью аналогичен процессу умножения десятичных чисел. На рисунке 5 подробно иллюстрируется решение примера на умножение двоичных чисел 111 и 101.

Десятичные числа		Двоичные числа
7	111	Множимое
$\times 5$	$\times 101$	Множитель
35	111	Первое частичное произведение
	+ 000	Второе частичное произведение
	111	Третье частичное произведение
	100011	Полное произведение

Рисунок 5 – Пример на умножение двоичных чисел

Сначала множимое 111 умножается на значение разряда единиц множителя. В результате получается первое частичное произведение, равное 111. Затем множимое умножается на значение разряда двоек множителя. Получаем второе частичное произведение 000. Обратите внимание, что нуль в младшем разряде второго частичного произведения отбрасывается. На третьем этапе множимое умножается на значение разряда четверок множителя. Получаем третье частичное произведение. Фактически оно равно 11100, но записывается как 111 (нули в двух младших разрядах опускаются). И наконец, первое, второе и третье частичные произведения складываются. Их сумма равна двоичному числу 100011. Это и есть полное произведение.

В общем случае [3] вычисление произведения $P (p_{2n-1}p_{2n-2} \dots p_1p_0)$ двух n -разрядных двоичных чисел без знака множимого (Мн) $A (a_{n-1}a_{n-2} \dots a_1a_0)$ и множителя (Мт) $B (b_{n-1}b_{n-2} \dots b_1b_0)$ сводится к формированию *частичных произведений* (ЧП) W_i по одному на каждую цифру множителя, с последующим суммированием полученных ЧП. Перед суммированием каждое частичное произведение должно быть сдвинуто на один разряд относительно предыдущего согласно весу цифры множителя, которой это ЧП соответствует (рисунок 6). Перемножение двух n -разрядных двоичных чисел $P = A \times B$ приводит к получению результата, содержащего $2n$ разрядов.

$a_3 \ a_2 \ a_1 \ a_0$	A
$\times b_3 \ b_2 \ b_1 \ b_0$	B
$w_{03} w_{02} w_{01} w_{00}$	$W_0 = Ab_0 \times 2^0$
+ $w_{13} w_{12} w_{11} w_{10}$	$W_1 = Ab_1 \times 2^1$
$w_{23} w_{22} w_{21} w_{20}$	$W_2 = Ab_2 \times 2^2$
$w_{33} w_{32} w_{31} w_{30}$	$W_3 = Ab_3 \times 2^3$
$p_7 \ p_6 \ p_5 \ p_4 \ p_3 \ p_2 \ p_1 \ p_0$	P

Рисунок 6 – Общая схема умножения для $n = 4$

Поскольку сомножителями являются двоичные числа, вычисление ЧП упрощается – если цифра множителя b_i равна 0, то W_i тоже равно 0, а при $b_i = 1$ частичное произведение равно множимому $W_i = A$.



Пример 15. Умножить по первому методу два целых числа без знака:
 $A = 1111, B = 101$ ($n = 4$).

Решение. Умножение с младших разрядов Мт со сдвигом Мн влево.

Шаг	Мт	Мн и СЧП	Действие
0	0101	00000000	$СЧП_0 = 0$
1	010 <u>1</u>	00001111 00001111 00011110	$ЧП_0 = Ab_0 = A$ $СЧП_1 = СЧП_0 + ЧП_0$ $A2^1$ (сдвиг Мн влево)
2	01 <u>0</u> 1	00000000 00001111 00111100	$ЧП_1 = A2^1 b_1 = 0$ $СЧП_2 = СЧП_1 + ЧП_1$ $A2^2$ (сдвиг Мн влево)
3	0 <u>1</u> 01	00111100 01001011 01111000	$ЧП_2 = A2^2 b_2 = A2^2$ $СЧП_3 = СЧП_2 + ЧП_2$ $A2^3$ (сдвиг Мн влево)
4	<u>0</u> 101	00000000 01001011 11110000	$ЧП_3 = A2^3 b_3 = 0$ $P = СЧП_4 = СЧП_3 + ЧП_3$ $A2^4$ (сдвиг Мн влево)

Решение. Умножение со старших разрядов Мт со сдвигом СЧП влево.

Шаг	Мт	Мн и СЧП	Действие
0	0101	00000000	$СЧП_0 = 0$
1	<u>0</u> 101	00000000 0000 00000000	$СЧП_0 2^1$ (сдвиг СЧП влево) $ЧП_0 = Ab_3 = 0$ $СЧП_1 = СЧП_0 2^1 + ЧП_0$
2	0 <u>1</u> 01	00000000 1111 00001111	$СЧП_1 2^1$ (сдвиг СЧП влево) $ЧП_1 = Ab_2 = A$ $СЧП_2 = СЧП_1 2^1 + ЧП_1$
3	01 <u>0</u> 1	00011110 0000 00011110	$СЧП_2 2^1$ (сдвиг СЧП влево) $ЧП_2 = Ab_1 = 0$ $СЧП_3 = СЧП_2 2^1 + ЧП_2$
4	010 <u>1</u>	00111100 1111 01001011	$СЧП_3 2^1$ (сдвиг СЧП влево) $ЧП_3 = Ab_0 = A$ $P = СЧП_4 = СЧП_3 2^1 + ЧП_3$

Пример 17. Умножить по третьему методу два целых числа без знака:
 $A = 1111, B = 101$ ($n = 4$).

Решение. Умножение со старших разрядов Мт со сдвигом Мн вправо:
 $V = A2^4 = 11110000$.

Шаг	Мт	Мн и СЧП	Действие
0	0101	00000000	$СЧП_0 = 0$
1	0101	01111000 00000000 00000000	$V2^{-1}$ (сдвиг Мн вправо) $ЧП_0 = V2^{-1}b_3 = 0$ $СЧП_1 = СЧП_0 + ЧП_0$
2	0101	00111100 00111100 00111100	$V2^{-2}$ (сдвиг Мн вправо) $ЧП_1 = V2^{-2}b_2 = V2^{-2}$ $СЧП_2 = СЧП_1 + ЧП_1$
3	0101	00011110 00000000 00111100	$V2^{-3}$ (сдвиг Мн вправо) $ЧП_2 = V2^{-3}b_1 = 0$ $СЧП_3 = СЧП_2 + ЧП_2$
4	0101	00001111 00001111 01001011	$V2^{-4}$ (сдвиг Мн вправо) $ЧП_3 = V2^{-4}b_0 = V2^{-4}$ $P = СЧП_4 = СЧП_3 + ЧП_3$

Пример 18. Умножить по четвертому методу два целых числа без знака:
 $A = 111, B = 101$ ($n = 3$).

Решение. Умножение с младших разрядов Мт со сдвигом СЧП вправо:
 $V = A2^4 = 111000$.

Шаг	Мт	Мн и СЧП	Действие
0	101	000000	$СЧП_0 = 0$
1	101	111 111000 011100	$ЧП_0 = Vb_0 = V$ $СЧП_1 = СЧП_0 + ЧП_0$ $СЧП_12^{-1}$ (сдвиг СЧП вправо)
2	101	000 011100 001110	$ЧП_1 = Vb_1 = 0$ $СЧП_2 = СЧП_12^{-1} + ЧП_1$ $СЧП_22^{-1}$ (сдвиг СЧП вправо)
3	101	111 1000110 100011	$ЧП_2 = Vb_2 = V$ $СЧП_3 = СЧП_22^{-1} + ЧП_2$ $P = СЧП_32^{-1}$ (сдвиг СЧП вправо)

*

24. Умножение двоичных чисел со знаком. Умножение чисел в дополнительном коде

Таким образом, алгоритм умножения чисел со знаком можно сформулировать следующим образом:

- 1) множимое и множитель представляются в дополнительном коде;

35

2) исходное значение суммы частичных произведений $СЧП_0$ принимается равным нулю;

3) анализируется очередной значащий разряд множителя b_i (анализ начинается с младшего разряда). Если $b_i = 1$, то к $СЧП_i$ прибавляется множимое, сдвинутое влево на i разрядов $ЧП_i = V = A2^i$ в том коде, в котором оно представлено. Если $b_i = 0$, то прибавление не производится. В результате получается очередная сумма частичных произведений $СЧП_{i+1}$;

4) производится арифметический сдвиг суммы частичных произведений вправо на один разряд $СЧП_{i+1}2^{-1}$;

5) пункты 3 и 4 последовательно повторяются для всех значащих разрядов множителя. После анализа старшего значащего разряда b_{n-1} множителя осуществляются последнее сложение $ЧП_{n-1} = V$ с $СЧП_{n-1}2^{-1}$ (если $b_{n-1} = 1$) и последний сдвиг $СЧП_n$ вправо;

6) анализируется знаковый разряд множителя. Если он равен единице (множитель отрицательный), то из $СЧП_n2^{-1}$ вычитается $V = A2^n$ (прибавляется V с обратным знаком, представленное в дополнительном коде, $[-V]_{\text{доп}} = [-A]_{\text{доп}}2^n$). Если знаковый разряд множителя равен нулю (множитель положительный), то вычитание не производится. В результате получается произведение, представленное в дополнительном коде.

Пример 19. Умножить два целых числа со знаком: $A = -13$, $B = +10$ ($n = 4$).

Решение. $[A]_{\text{доп}} = 1.0011$; $[B]_{\text{доп}} = 0.1010$; $V = [A]_{\text{доп}}2^4 = 1.0011 \cdot 2^4$.

Шаг	МТ	Мн и СЧП	Действие
0	1010	0.00000000	$СЧП_0 = 0$
1	1010	0.0000 0.00000000 0.00000000	$ЧП_0 = Vb_0 = 0$ $СЧП_1 = СЧП_0 + ЧП_0$ $СЧП_12^{-1}$ (сдвиг СЧП вправо)
2	1010	1.0011 1.00110000 1.10011000	$ЧП_1 = Vb_1 = V$ $СЧП_2 = СЧП_12^{-1} + ЧП_1$ $СЧП_22^{-1}$ (сдвиг СЧП вправо)
3	1010	0.0000 1.10011000 1.11001100	$ЧП_2 = Vb_2 = 0$ $СЧП_3 = СЧП_22^{-1} + ЧП_2$ $СЧП_32^{-1}$ (сдвиг СЧП вправо)
4	1010	1.0011 1.01111100 1.01111110 <i>отбрасывается</i>	$ЧП_3 = Vb_3 = V$ $СЧП_4 = СЧП_32^{-1} + ЧП_3$ $[P]_{\text{доп}} = СЧП_42^{-1}$ (сдвиг СЧП вправо)
		1.10000010	$[P]_{\text{пр}}$

Обратите внимание, что в соответствии с правилами сложения в дополнительных кодах возникающая в примере 19 на шаге 4 в результате сложения единица переноса из знакового разряда отбрасывается.

Пример 20. Умножить два целых числа со знаком: $A = +13$, $B = -10$ ($n = 4$).

Решение

$$[A]_{\text{доп}} = 0.1101; [B]_{\text{доп}} = 1.0110; V = [A]_{\text{доп}} 2^4 = 0.1101 \cdot 2^4.$$

$$\text{Величина коррекции } -A2^4 = [-A]_{\text{доп}} 2^4 = 1.0011 \cdot 2^4.$$

Шаг	МТ	Мн и СЧП	Действие
0	0110	0.00000000	$\text{СЧП}_0 = 0$
1	0110	0.0000 0.00000000 0.00000000	$\text{ЧП}_0 = Vb_0 = 0$ $\text{СЧП}_1 = \text{СЧП}_0 + \text{ЧП}_0$ $\text{СЧП}_1 2^{-1}$ (сдвиг СЧП вправо)
2	0110	0.1101 0.11010000 0.01101000	$\text{ЧП}_1 = Vb_1 = V$ $\text{СЧП}_2 = \text{СЧП}_1 2^{-1} + \text{ЧП}_1$ $\text{СЧП}_2 2^{-1}$ (сдвиг СЧП вправо)
3	0110	0.1101 1.00111000 0.10011100	$\text{ЧП}_2 = Vb_2 = 0$ $\text{СЧП}_3 = \text{СЧП}_2 2^{-1} + \text{ЧП}_2$ $\text{СЧП}_3 2^{-1}$ (сдвиг СЧП вправо)
4	0110	0.0000 0.10011100 0.01001110	$\text{ЧП}_3 = Vb_3 = V$ $\text{СЧП}_4 = \text{СЧП}_3 2^{-1} + \text{ЧП}_3$ $P' = \text{СЧП}_4 2^{-1}$ (сдвиг СЧП вправо)
Коррек- ция		1.0011 1.01111110	$[-A]_{\text{доп}} 2^4$ $[P]_{\text{доп}}$
		1.10000010	$[P]_{\text{пр}}$

Ответ. $P = -130$.

Пример 21. Умножить два целых числа со знаком: $A = -13$, $B = -10$ ($n = 4$).

Решение

$$[A]_{\text{доп}} = 1.0011; [B]_{\text{доп}} = 1.0110; V = [A]_{\text{доп}} 2^4 = 1.0011 \cdot 2^4.$$

$$\text{Величина коррекции } -A2^4 = [-A]_{\text{доп}} 2^4 = 0.1101 \cdot 2^4.$$

Шаг	Мт	Мн и СЧП	Действие
0	0110	0.00000000	$СЧП_0 = 0$
1	011 $\boxed{0}$	0.0000 0.00000000 0.00000000	$ЧП_0 = Vb_0 = 0$ $СЧП_1 = СЧП_0 + ЧП_0$ $СЧП_1 2^{-1}$ (сдвиг СЧП вправо)
2	01 $\boxed{1}$ 0	1.0011 1.00110000 1.10011000	$ЧП_1 = Vb_1 = V$ $СЧП_2 = СЧП_1 2^{-1} + ЧП_1$ $СЧП_2 2^{-1}$ (сдвиг СЧП вправо)
3	0 $\boxed{1}$ 10	1.0011 $\boxed{1}$ 0.11001000 1.01100100 <i>отбрасывается</i>	$ЧП_2 = Vb_2 = 0$ $СЧП_3 = СЧП_2 2^{-1} + ЧП_2$ $СЧП_3 2^{-1}$ (сдвиг СЧП вправо)
4	$\boxed{0}$ 110	0.0000 1.01100100 1.10110010	$ЧП_3 = Vb_3 = V$ $СЧП_4 = СЧП_3 2^{-1} + ЧП_3$ $P' = СЧП_4 2^{-1}$ (сдвиг СЧП вправо)
Коррек- ция		0.1101 $\boxed{1}$ 0.10000010 <i>отбрасывается</i>	$[-A]_{\text{доп}} 2^4$ $[P]_{\text{доп}} = [P]_{\text{пр}}$

Ответ. $P = +130$.

25. Логические методы ускорения умножения: алгоритм Бута

Логические подходы к ускорению умножения можно подразделить на две группы:

- применение избыточных систем счисления;
- применение систем счисления с основанием большим двух.

Рассмотрим первую группу логических методов — умножение с использованием избыточных систем счисления. Наиболее известным и распространенным представителем этой группы является алгоритм Бута. В основе алгоритма Бута лежит следующее соотношение, характерное для последовательностей двоичных цифр:

$$2^m + 2^{m-1} + \dots + 2^k = 2^{m+1} - 2^k.$$

где m и k — номера крайних разрядов в группе из последовательных единиц. Например, $011110 = 2^5 - 2^1$. Это означает, что при наличии во множителе групп из нескольких единиц (комбинаций вида 011110), последовательное добавление к СЧП множимого с нарастающим весом (от 2^k до 2^m) можно заменить вычитанием из СЧП множимого с весом 2^k и прибавлением к СЧП множимого с весом 2^{m+1} .

Алгоритм Бута сводится к перекодированию множителя из системы $\{0, 1\}$ в избыточную систему $\{\bar{1}, 0, 1\}$. Такое перекодирование часто называют перекодированием Бута. В записи множителя в новой системе $\bar{1}$ означает добавление множимого к сумме частичных произведений, $\bar{1}$ — вычитание множимого и 0 не предполагает никаких действий. Во всех случаях после очередной итерации производится сдвиг множимого влево или суммы частичных произведений вправо. Реализация алгоритма предполагает последовательный в направлении справа налево анализ пар разрядов множителя — текущего b_i и предшествующего b_{i-1} . Для младшего разряда множителя ($i = 0$) считается, что предшествующий разряд равен 0 , т. е. имеет место пара $b_0 0$. На каждом шаге i ($i = 0, 1, \dots, n-1$) анализируется текущая комбинация $b_i b_{i-1}$.

Алгоритм предполагает три операции: сдвиг, сложение и вычитание. Комбинация 10 означает начало цепочки последовательных единиц, и в этом случае производится вычитание множимого из СЧП. Комбинация 01 соответствует завершению цепочки единиц, и в этом случае множимое прибавляется к СЧП. Комбинация 00 свидетельствует об отсутствии цепочки единиц, а 11 — о

нахождении внутри такой цепочки. В обоих случаях никакие арифметические операции не производятся. По завершении описанных действий осуществляется сдвиг множимого влево либо суммы частичных произведений вправо, и цикл повторяется для следующей пары разрядов множителя. Если последняя пара разрядов множителя $b_{n-1} b_{n-2}$ равна комбинациям 10 или 11 , должна быть выполнена коррекция результата, которая заключается в добавлении множимого к последней сумме частичных произведений.

Помимо сокращения числа сложений (вычитаний) этот алгоритм имеет еще одно достоинство: он в равной степени применим как к числам без знака, так

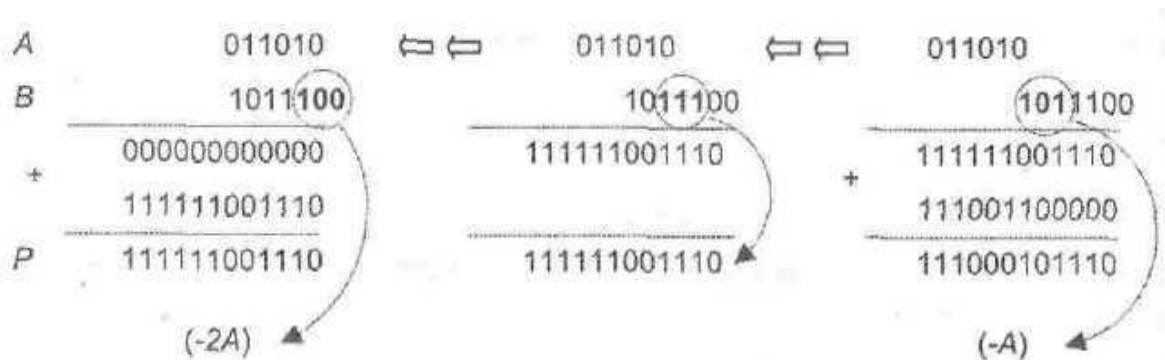
и к числам со знаком. Процесс умножения по алгоритму Бута чисел со знаком отличается тем, что знаковый разряд множителя участвует в анализе наравне со значащими разрядами. При этом, поскольку знаковый разряд участвует в процессе умножения, коррекцию результата производить не нужно.

26. Логические методы ускорения умножения: модифицированный алгоритм Бута, алгоритм Лемана

На практике большее распространение получила модификация алгоритма Бута, где количество операций сложения при любом сочетании единиц и нулей в множителе всегда равно $n/2$. В модифицированном алгоритме производится перекодировка цифр множителя из стандартной двоичной системы (0, 1) в избыточную систему (-2, -1, 0, 1, 2), где каждое число представляет собой коэффициент, на который умножается множимое перед добавлением к СЧП. Одновременно анализируются три разряда множителя $b_{i+1}b_i b_{i-1}$ (два текущих и старший разряд из предыдущей тройки) и, в зависимости от комбинации 0 и 1 в этих разрядах, выполняется прибавление или вычитание множимого, прибавление или вычитание удвоенного множимого, либо никакие действия не производятся

x_{i+1}	x_i	x_{i-1}	Код (-2 -1 0 1 2)	Выполняемые действия
0	0	0	0	Не выполнять никаких действий
0	0	1	1	Прибавить к СЧП множимое
0	1	0	1	Прибавить к СЧП множимое
0	1	1	2	Прибавить к СЧП удвоенное множимое
1	0	0	-2	Вычесть из СЧП удвоенное множимое
1	0	1	-1	Вычесть из СЧП удвоенное множимое
1	1	0	-1	Вычесть из СЧП удвоенное множимое
1	1	1	0	Не выполнять никаких действий

Алгоритм
Лемана



Еще большее сокращение количества сложений может дать модификация, предложенная Леманом [151]. Здесь, даже при наименее благоприятном сочетании цифр множителя, количество операций сложения не превышает величины $n/2$, а в среднем же оно составляет $n/3$. Суть модификации заключается в следующем:

- если две группы нулей разделены единицей, стоящей в k -й позиции, то вместо вычитания в k -й позиции и сложения в $(k + 1)$ -й позиции достаточно выполнить только сложение в k -й позиции;
- если две группы единиц разделены нулем, стоящим в k -й позиции, то вместо сложения в k -й позиции и вычитания в $(k + 1)$ -й позиции достаточно выполнить только вычитание в k -й позиции.

27. Логические методы ускорения умножения: обработка двух разрядов множителя за шаг

При умножении 2 разрядов за шаг уменьшается количество элементарных действий (сложение и вычитание), в следствии чего кол-во времени, затраченного на умножение, уменьшается в 2 раза. Ниже очень важная табличка. Она нужна для преобразования множителя. Сначала надо разбить множитель на пары (00, 01, 10, 11). Начинать преобразование надо с правой стороны. Для первой пары переноса из младшей нет (на всякий).

Анализируемая пара разрядов	Перенос из младшей пары	Преобразованная пара	Перенос в старшую пару
00	0	00	0
01	0	01	0
10	0	10	0
11	0	0 $\bar{1}$	1
00	1	01	0
01	1	10	0
10	1	0 $\bar{1}$	1
11	1	00	1

$$M_d = \begin{array}{cccc} \underline{11} & \underline{00} & \underline{01} & \underline{11} \\ \downarrow & & \downarrow & \downarrow \\ M_4^n = \begin{array}{cccc} \underline{01} & \underline{0\bar{1}} & \underline{00} & \underline{10} & \underline{0\bar{1}} \\ 1 & -1 & 0 & 2 & -1 \end{array} \quad \text{(перенос)}$$

т.к. есть перенос но нет пары, мы берём 00 и добавляем перенос

$$01 = \times 1$$

$$10 = \times 2$$

$$0\bar{1} = \times (-1)$$

$$00 = \times 0$$

Мы получили 5 пар $M_t(n)$ а дальше реализуем их как с обычным умножением (Пример ниже). Также неплохо было бы заранее посчитать $(-M_n)$ и $(2 \cdot M_n)$ в доп кодах.

Пример. $M_H = 0101$

$M_T = 11000111$

M_T разбиваем на пары и преобразуем согласно табл. 1.

$M_T^H = 010\bar{1}00100\bar{1}$ – преобразованный M_T .

Умножение будем осуществлять согласно алгоритму А.

$[-M_H]_{\text{доп}} = 1.1011$

$[2 M_H]_{\text{доп}} = 0.1010$

0.0000	Σ_0^H
+ <u>1.1011</u>	$\Pi_1^H = -M_H$
1.1011	Σ_1^H
1. 1 10 11	$\Sigma_1^H \cdot 2^{-2}$
+ <u>0.1010</u>	$\Pi_2^H = 2M_H$
0.1000 11	Σ_2^H
0. 00 10 0011	$\Sigma_2^H \cdot 2^{-2}$
0. 0000 100011	$\Sigma_3^H \cdot 2^{-2}$ ($\Sigma_2^H \cdot 2^{-4}$, так как $\Pi_3^H = 0$)
+ <u>1.1011</u>	$\Pi_4^H = -M_H$
1.1011 100011	Σ_4^H
1. 1 110 11100011	$\Sigma_4^H \cdot 2^{-2}$
+ <u>0.0101</u>	$\Pi_5^H = M_H$
0.0011 11100011	Σ_5^H
0. 0000 1111100011	$\Sigma_5^H \cdot 2^{-2}$

28. Деление чисел с фиксированной запятой. Методы деления двоичных чисел без знака

Алгоритм деления без восстановления остатка.

1. Выполняется пробное вычитание с формированием первого остатка $A_1 = [\text{Дм}]_{\text{доп}} + [-\text{Дт}]_{\text{доп}}$. Далее, если $A_1 < 0$, то в первый разряд, расположенный слева от запятой, заносится нуль (0), иначе – единица (1) – что является признаком переполнения и осуществляется переход к п. 5.

2. Формирование очередного остатка. Если $A_i < 0$, то $A_{i+1} = A_i \cdot 2 + [\text{Дт}]_{\text{доп}}$, иначе $A_{i+1} = A_i \cdot 2 + [-\text{Дт}]_{\text{доп}}$.

3. Если $A_{i+1} < 0$, то в очередной разряд частного справа от запятой записывается нуль ($\text{Чт}(n) = 0$), иначе записывается единица ($\text{Чт}(n) = 1$).

4. Если достигнута заданная точность частного или получен нулевой остаток A_{i+1} , то процесс деления окончен и осуществляется переход к п. 5, иначе возвращаемся к п. 2 алгоритма.

5. Конец алгоритма.

Пример.

Дм = 0,1011	0,1011	Дм (A_0)
Дт = 0,1101	+ 1,0011	$[-\text{Дт}]_{\text{доп}}$
$[-\text{Дт}]_{\text{доп}} = 1,0011$	<u>1,1110</u>	A_1
	1,1100	$A_1 \cdot 2$
	+ 0,1101	+ Дт
	<u>0,1001</u>	A_2
	1,0010	$A_2 \cdot 2$
	+ 1,0011	$[-\text{Дт}]_{\text{доп}}$
	<u>0,0101</u>	A_3
Чт = 0, 1 1 0 ...	0,1010	$A_3 \cdot 2$
	+ 1,0011	$[-\text{Дт}]_{\text{доп}}$
	<u>1,1101</u>	A_4

29. Деление двоичных чисел со знаком. Деление чисел в дополнительном коде

Деление чисел со знаком, как и умножение, может быть выполнено путем перехода к абсолютным значениям делимого и делителя с последующим присвоением частному знака плюс при совпадающих знаках делимого и делителя, либо минус – в противном случае. Остаток всегда имеет знак делимого.

Деление чисел, представленных в дополнительном коде, можно осуществлять, не переходя к модулям. При этом алгоритм деления видоизменяется в соответствии с особенностями использования дополнительного кода. Рассмотрим эти изменения для метода деления без восстановления остатка.

Так как делимое и делитель не обязательно имеют одинаковые знаки, то действия с частичным остатком (прибавление или вычитание Дт) зависят от знаков остатка и делителя и определяются содержимым таблицы.

Знак остатка	Знак делителя	Действие
+	+	Вычитание делителя
+	-	Прибавление делителя
-	+	Прибавление делителя
-	-	Вычитание делителя

Если знак остатка совпадает со знаком делителя, то очередной разряд частного равен 1, в противном случае – 0.

После завершения алгоритма может потребоваться коррекция частного, которая заключается в увеличении частного на единицу. Такая коррекция должна производиться в следующих случаях:

- 1) $Дм > 0$ и $Дт < 0$;
- 2) $Дм < 0$ и $Дт > 0$ при не нулевом остатке от деления;
- 3) $Дм < 0$ и $Дт < 0$ при нулевом остатке от деления.

Алгоритм:

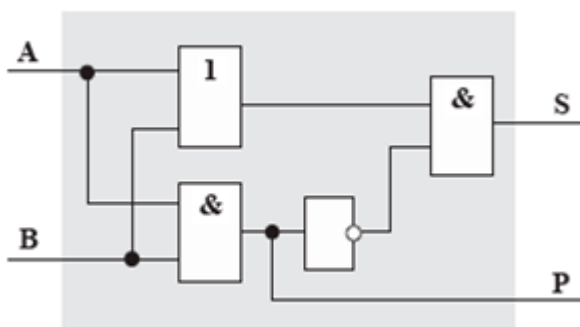
1. Выполняется пробное вычитание: если знак $Дм$ ¹ знаку $Дт$, то первый остаток $A_1 = [Дм]_{доп} + [Дт]_{доп}$, иначе $A_1 = [Дм]_{доп} + [-Дт]_{доп}$. Далее формируется первый разряд, расположенный слева от запятой - ноль (0), если знак A_1 ¹ знаку $Дт$, иначе единица (1).
2. Формирование очередного остатка. Если знак A_i ¹ знаку $Дт$, то $A_{i+1} = A_i \cdot 2 + [Дт]_{доп}$, иначе $A_{i+1} = A_i \cdot 2 + [-Дт]_{доп}$.
3. Если знак A_{i+1} ¹ знаку $Дт$, то в очередной разряд частного справа от запятой заносится ноль ($Чт(n)=0$), иначе - единица ($Чт(n)=1$).
4. Если достигнута заданная точность частного или получен нулевой остаток A_{i+1} , то процесс деления окончен, иначе переходим к пункту 2 алгоритма.

	Делимое 41_{10}	0 0 1 0 1 0 0 1	0 1 1 1	Делитель 7_{10}
Исходное значение ЧО		0 0 1 0 1 0 0 1	0 1 0 1	Частное 5_{10}
Сдвиг ЧО влево		0 0 1 0 1 0 0 1 0	↑	
Вычитание делителя	-	0 1 1 1	↑	
Результат < 0		Ⓢ 1 1 1 0 0 0 1 0	↑	
Восстановление ЧО	+	0 1 1 1	↑	
Восстановленный ЧО		0 0 1 0 1 0 0 1 0		
Сдвиг ЧО влево		0 0 1 0 1 0 0 1 0 0		
Вычитание делителя	-	0 1 1 1		
Результат > 0		Ⓢ 0 0 0 1 1 0 1 0 0		
Сдвиг ЧО влево		0 0 0 0 1 1 0 1 0 0 0		
Вычитание делителя	-	0 1 1 1		
Результат < 0		Ⓢ 1 1 1 1 1 1 1 0 0 0		
Восстановление ЧО	+	0 1 1 1		
Восстановленный ЧО		0 0 0 0 1 1 0 1 0 0 0		
Сдвиг ЧО влево		0 0 0 0 1 1 0 1 0 0 0 0		
Вычитание делителя	-	0 1 1 1		
Результат > 0		Ⓢ 0 0 0 0 0 0 1 1 0 1 0 0		
		0 0 1 1		Остаток 6_{10}

30. Одноразрядные сумматоры

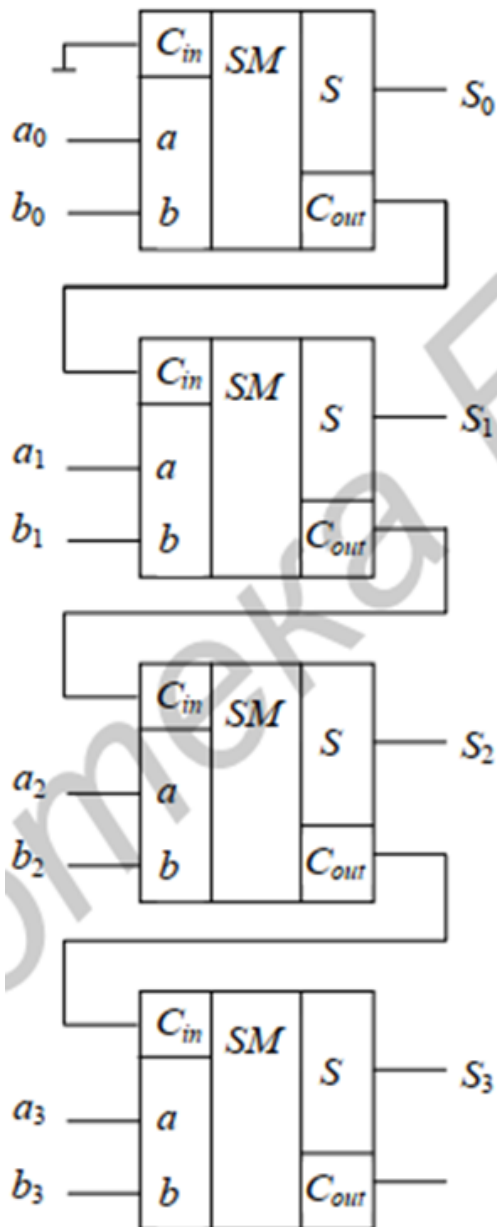
Суммирование двух чисел происходит поразрядно, с определением цифры в данном разряде и сигнала переноса в соседний, более старший разряд.

При суммировании младших разрядов требуется одноразрядный сумматор с двумя входами, поскольку входного сигнала переноса в этом разряде нет. Такие сумматоры, в отличие от сумматоров с тремя входами, называют полусумматорами



31. Параллельный сумматор

Параллельный сумматор в простейшем случае представляет собой n одноразрядных сумматоров, последовательно (от младших разрядов к старшим) соединенных цепями переноса. Сумматор имеет 3 входа, переменные a и b , C_{in} и C_{out} – перенос и выход S . Хотя сумматор и называется параллельным, на самом деле все разряды обрабатываются не одновременно, а только после формирования переноса для данного разряда.



32. Накапливающий сумматор

Накапливающий сумматор представляет собой сочетание комбинационного сумматора и регистра.

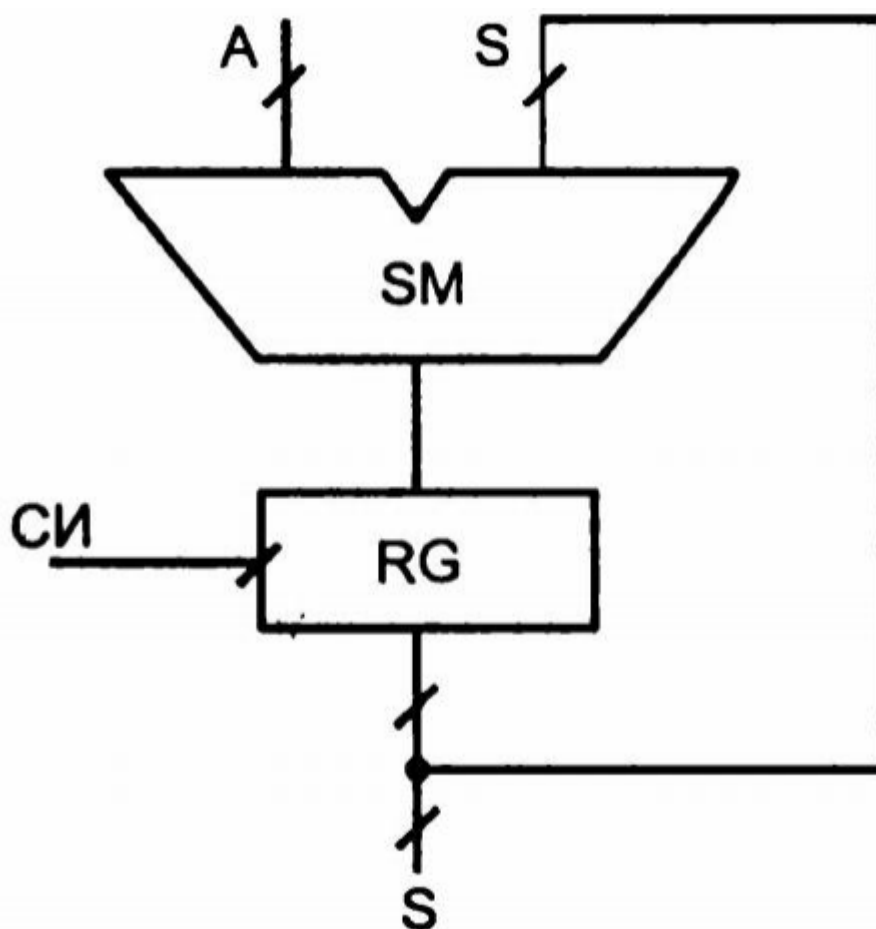
Работает по формуле, согласно которой к содержимому сумматора добавляется очередное слагаемое и результат замещает старое значение суммы.

Очередное прибавление слагаемого тактируется синхроимпульсами СИ.

Учитывая особенности функционирования, накапливающие сумматоры называют иногда аккумуляторами.

Накапливающий сумматор состоит из сумматора и выходного регистра, охваченных обратной связью.

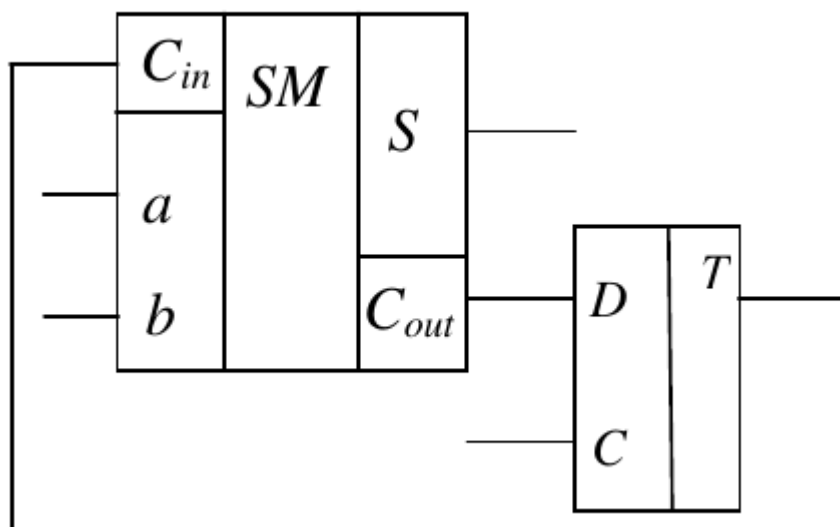
Структура накапливающего сумматора :



33. Последовательный сумматор

Для последовательного выполнения операции сложения (разряд за разрядом) используется один полный одноразрядный сумматор. Он имеет два входа (a и b) на каждый из которых последовательно, начиная с младшего, подаются разряды слагаемых (по одному за такт). С выхода снимается значение соответствующего разряда суммы. Последнее

определяется не только текущими значениями разрядов слагаемых, но и переносом из предыдущего разряда C_{in} .



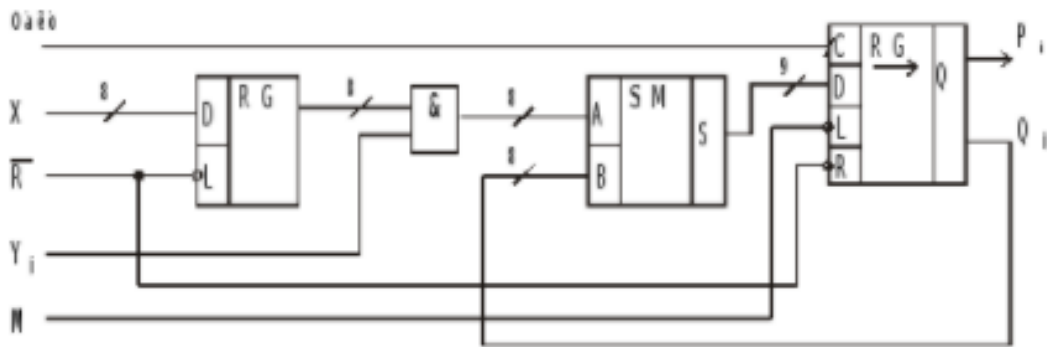
34. Арифметико-логические устройства

Арифметико-логическое устройство (АЛУ) — функциональная часть ЭВМ, которая выполняет логические и арифметические действия, необходимые для переработки информации, хранящейся в памяти.

В современных вычислительных устройствах основным исполнительным элементом является процессор (П) или микропроцессор (МП), который содержит в себе АЛУ.

35. Последовательные умножители

В последовательных умножителях множимое X_n записывается в регистр, а разряды множителя Y_m поступают последовательно, начиная с младшего разряда. В результате последовательно формулируются частичные произведения и происходит их арифметическое накопление. На рис. 87 приведена схема последовательного умножителя ИП9 — умножитель 8×1 . Сигнал M определяет режим работы, При $M = 0$ происходит загрузка числа X , при $M = 1$ выполняется операция сдвига и поразрядное умножение. Множитель Y может имеет произвольную разрядность.



36. Матричные умножители

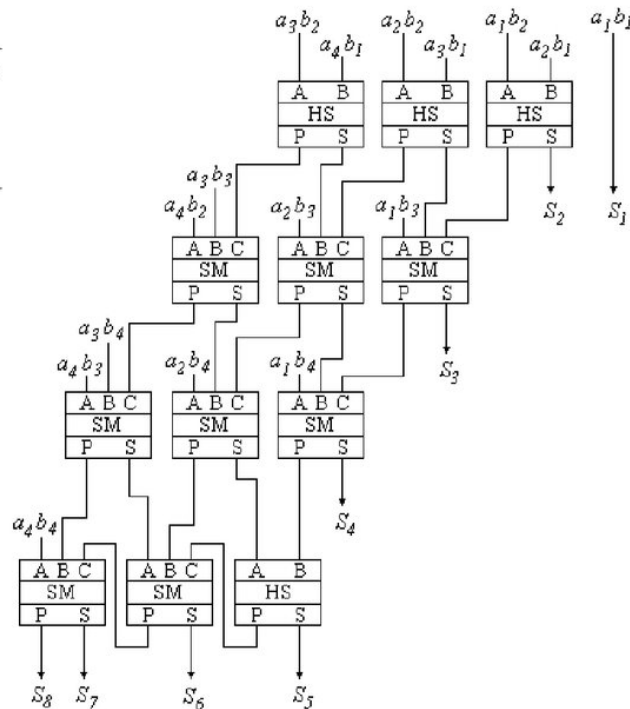
Матричный умножитель представляет собой специализированный многовходовой сумматор, позволяющий одновременно суммировать все частичные произведения.

(P.S.: по факту это наше привычное умножение в столбик больших чисел, где потом элементы сносятся вниз и складываются в конечное число)

Матричный умножитель

$$\begin{array}{r} \times \begin{array}{l} A = a_4 \quad a_3 \quad a_2 \quad a_1 \\ B = b_4 \quad b_3 \quad b_2 \quad b_1 \end{array} \\ \hline \begin{array}{l} a_4 b_1 \quad a_3 b_1 \quad a_2 b_1 \quad a_1 b_1 \\ + a_4 b_2 \quad a_3 b_2 \quad a_2 b_2 \quad a_1 b_2 \\ + a_4 b_3 \quad a_3 b_3 \quad a_2 b_3 \quad a_1 b_3 \\ + a_4 b_4 \quad a_3 b_4 \quad a_2 b_4 \quad a_1 b_4 \end{array} \\ \hline C = \begin{array}{cccccccc} s_8 & s_7 & s_6 & s_5 & s_4 & s_3 & s_2 & s_1 \end{array} \end{array}$$

$$t = 2 \cdot t_{HS} + t_{SM} \cdot (n_A + n_B - 4)$$



37. Сложение и вычитание чисел с плавающей запятой

Операции сложения и вычитания выполняются одинаково, но в случае вычитания необходимо изменить знак второго операнда на противоположный.

Чтобы сложить два числа с плавающей запятой, нужно, прежде всего, произвести выравнивание их порядков.

Алгоритм сложения двух чисел с произвольными знаками сводится к выполнению следующей последовательности действий:

1. производится сравнение порядков p_A и p_B слагаемых A и B путем вычитания порядка p_B второго слагаемого из порядка p_A первого слагаемого или наоборот. Для вычитания порядков знак p_B меняется на противоположный и далее порядки складываются по правилу сложения целых чисел, при этом отрицательный порядок вступает в операцию сложения в дополнительном или обратном коде.

Разность $p = p_A - p_B$ указывает, на сколько разрядов надо сдвинуть вправо мантиссу числа с меньшим порядком.

2. мантисса числа с меньшим порядком сдвигается вправо на p разрядов. Если в процессе сдвига мантисса обращается в 0, в качестве результата операции берется другой операнд

3. выполняется сложение мантисс чисел с равными порядками по правилу сложения правильных дробей, при этом отрицательная мантисса вступает в операцию сложения в дополнительном или обратном коде. В результате выполнения операции сложения может получиться нулевое значение мантиссы. В этом случае имеет место ситуация потери значимости мантиссы, и результат операции принимается равным нулю

4. если при сложении мантисс произошло переполнение (в знаковом разряде оказалась старшая цифра суммы, т.е. произошло нарушение нормализации влево), производится нормализация путем сдвига мантиссы суммы вместе со знаковым разрядом вправо на один разряд с увеличением порядка на единицу, при этом в знаковый разряд мантиссы суммы записывается знак результата, т.е. выполняется арифметический сдвиг

В процессе нормализации возможно переполнение порядка. В этом случае операция прекращается и формируется признак переполнения

Если же после сложения мантисса суммы стала меньше 0,5 (в абсолютном значении мантиссы суммы после запятой оказались нули, т.е. произошло нарушение нормализации вправо), мантисса суммы сдвигается влево на количество разрядов, занятых нулями

Одновременно из порядка p результата вычитается число, равное этому количеству нулей.

38. Умножение чисел с плавающей запятой

При умножении чисел A и B с плавающей запятой мантисса произведения M_p определяется путем перемножения мантисс сомножителей $M_p = M_A M_B$.

Порядок произведения p_p определяется путем сложения порядков сомножителей $p_p = p_A + p_B$.

Если используется смещенный порядок, то в полученной сумме будет содержаться удвоенное смещение, поэтому из нее необходимо вычесть величину смещения.

Результатом действий с порядками может стать как переполнение порядка, так и потеря значимости.

Перемножение мантисс сомножителей выполняется по правилу умножения правильных дробей.

Знак произведения определяется путем суммирования по модулю два знаковых разрядов мантисс.

Поскольку мантиссы сомножителей нормализованы, абсолютное значение мантиссы M_p заключено в следующих пределах: $0,01_{(2)} < |M_p| < 1$, т.е. может иметь место нарушение нормализации вправо не более чем на один разряд

После перемножения мантисс сомножителей результат при необходимости нормализуется путем сдвига значащих разрядов мантиссы M_p на один разряд влево и вычитания единицы из порядка p_p

При нормализации результата возможно исчезновение порядка (потеря значимости порядка)

39. Деление чисел с плавающей запятой

При делении чисел, представленных в форме с плавающей запятой, частное определяется как деление делимого D на делитель d , а порядок - как разность порядков делимого и делителя. Деление мантисс чисел производится по тем же правилам, что и деление чисел с фиксированной запятой.

Порядок действий	Вычисления	Частное
1. Определение порядка результата, равного разности порядков делимого и делителя	$p = p_D - p_d = 1001 - 100 = 101$	-
2. Получение кодов мантисс делителя	$M_{d_{\text{н}}} = 0,1001$ $-M_{d_{\text{н}}} = 1,0111$	
Деление мантисс. Вычислительный процесс продолжается до тех пор, пока младший разряд ЧО, отмеченный звездочкой (*), не совпадет с младшим разрядом делителя d	1) $M_{D_{\text{н}}} = 0,0000\ 10001*$ $M_{d_{\text{н}}} = 1,0111$ $\underline{qO_1 = 1,0111\ 10001*} < 0$ $\underline{qO} = 0,1111\ 0001*0$ 2) $+$ $M_{d_{\text{н}}} = 0,1001$ $\underline{qO_2 = 1,1000\ 0001*} < 0$ $\underline{qO} = 1,0000\ 001*00$ 3) $+$ $M_{d_{\text{н}}} = 0,1001$ $\underline{qO_3 = 1,1001\ 001*} < 0$ $\underline{qO} = 1,0010\ 01*000$ 4) $+$ $M_{d_{\text{н}}} = 0,1001$ $\underline{qO_4 = 1,1011\ 01*} < 0$ $\underline{qO} = 1,0110\ 1*0000$ 5) $+$ $M_{d_{\text{н}}} = 0,1001$ $\underline{qO_5 = 1,1111*} < 0$ $\underline{qO} = 1,1111*00000$ 6) $+$ $M_{d_{\text{н}}} = 0,1001$ $\underline{qO_6 = 0,1000\ 00000} > 0$ остаток	0, 0,0 0,00 0,000 0,0000 0,00001

Деление чисел с плавающей запятой

Если числа A и B заданы в нормальной форме, то их частное: $Y=A:B=(a:b)2^{m_1-m_2}$. Поэтому операция деления выполняется в пять этапов:

- 1) определение знака частного путем сложения по mod2 знаковых цифр мантисс операндов.
- 2) деление модулей мантисс операндов по правилам деления чисел с фиксированной запятой;
- 3) определение порядка частного путем вычитания порядка делителя из порядка делимого;
- 4) нормализация результата и его округление;
- 5) Присвоение знака мантиссе результата.

При делении нормализованных чисел денормализация результата возможна только влево и только на один разряд - это обусловлено тем, что мантисса любого нормализованного числа

$$2^{-1} \leq |a| < 2^{-n}$$

Тогда наименьшая и наибольшая возможные величины мантиссы частного равны:

$$|y_{\min}| 2^{-1} > 2^{-1}; |y_{\max}| 2^{-n} = 2^{-2-(n+1)} < 2^{-n-1}$$

$$2^{-n-1}$$

Поэтому на четвертом этапе может возникнуть необходимость нормализации мантиссы частного путем ее сдвига вправо на один разряд и увеличение порядка частного на 1. Если же перед делением сдвинуть делимое на один разряд вправо, то на 4-м этапе может потребоваться нормализация результата влево на 1 разряд.

При выполнении третьего этапа порядок частного может оказаться больше допустимого, т.е. произойдет переполнение разрядной сетки. Может быть так же получен порядок частного, который меньше допустимого. Если на втором этапе вычислено частное $|y| > 1$, то на четвертом этапе при сдвиге частного вправо его порядок должен быть увеличен на единицу. При этом становится окончательно ясно:

- 1) возникло ли переполнение разрядной сетки порядка;
- 2) будет ли порядок частного меньше допустимого значения. Если имеет место исчезновение порядка, то результат заменяется машинным путем.

40. Выполнение операции сложения в двоично-десятичном коде

При выполнении операций над отмеченными кодами возможны следующие особенности:

- наличие разрешенных и запрещенных комбинаций, свидетельствующих о правильности результата или необходимости его коррекции;
- при сложении тетрад возможен потетрадный (16 единиц), а не поразрядный (10 единиц) перенос, что также требует корректировки результата.

При сложении чисел в *BCD*-коде возможны 2 случая:

1. $(a + b) \leq 9$. В этом случае если действия выполняются по правилам двоичной арифметики, то величина получаемой суммы не превышает девяти и коррекция результата не требуется.
2. Если $10 \leq (a + b) \leq 16$ (т. е. возникла запрещенная комбинация), то выполняется коррекция путем добавления 6 (0110) к тетраде.

Пример. Необходимо сложить числа: $A = 169$ и $B = 378$ в *BCD*-коде.

$A = 169$	$A = 0.0001\ 0110\ 1001$
$+ B = 378$	$+ B = 0.0011\ 0111\ 1000$
$A + B = 547$	$A + B = 0.0101\ 1110\ 0001$
	$\underline{\hspace{1.5cm}0110\ 0110\hspace{0.5cm}}$
	$0.0101\ 0100\ 0111$
	$\downarrow$
	перенос игнорируется

41. Выполнение операции вычитания в двоично-десятичном коде

Отрицательные *BCD*-коды должны представляться в прямом, обратном или дополнительном коде. Особенностью *BCD*-кодов является то, что инверсия тетрады означает дополнение до 15, а для соответствующей десятичной цифры необходимо дополнить до 9. Следовательно, нужно убрать разницу. Один из

приемов формирования обратного *BCD*-кода состоит в добавлении во все тетрады отрицательного числа 0110, затем их инверсии.

$$\begin{array}{rcl}
 a - b \geq 0 & & \\
 a = 7 & 0.0111 & [a]_{\text{обр}} \\
 b = \underline{-3} & +\underline{1.1100} & [b]_{\text{обр}} \\
 4 & 10.0011 & \\
 & \xrightarrow{\quad} 1 & \\
 & \underline{\quad} & \\
 & 0.0100 &
 \end{array}$$

При образовании инверсии отрицательной тетрады в нее добавляются 15 единиц. Эти 15 единиц находятся и в сумме. Но благодаря шестнадцатеричному переносу из тетрады суммы уходит 16 единиц (одна единица восстанавливается добавлением по цепи циклического переноса).

$$\begin{array}{rcl}
 2. a - b < 0 & & \\
 a = 3 & 0.0011 & [a]_{\text{обр}} \\
 b = \underline{-7} & +\underline{1.1000} & [b]_{\text{обр}} \\
 -4 & 1.1011 & \\
 & 0.0100 &
 \end{array}$$

Здесь, как и в предыдущем примере, в тетраде суммы пятнадцать лишних единиц. При переходе от инверсной формы к прямой лишние единицы уничтожаются сами собой. Это то же самое, что от значащей части суммы вычесть пятнадцать: $1011 - 1111 = 0100$.

42. Сложение в двоично-десятичном коде чисел со знаком

Двоично-десятичное число состоит из тетрад (4 разряда)

Десятичная цифра	Код 8421		
0	0000		
1	0001		
2	0010	Десятичное число	Двоично-десятичное число
3	0011	8 0 7 2	1000 0000 0111 0010
4	0100		
5	0101		
6	0110		
7	0111		
8	1000		
9	1001		

Чтобы представить знак числа в BCD, число **0000** используется для представления положительного числа, а **1001** используется для представления отрицательного числа. Остальные 14 комбинаций - недопустимые знаки. (-58 = **1001** 0101 1000).

При переводе в дополнительный код + или - в виде **0000** или **1001** не изменяется. ((-58)доп = **1001** 0100 0010)

*Преобразование в доп. код описано в 41

Итак, есть 4 варианта сложения:

*Если у чисел различное кол-во цифр, то недостающие представляются в виде 0 (1053-2 как 1053-0002).

1) А и В оба положительны.

Идет просто сложение 2х чисел по правилу **40**. Только впереди рисуется + (**0000**).

2) А и В оба отрицательны.

Отрицательные числа остаются в прямом коде, т.к. результат сложения 2х дополнительных такой же как и при прямых кодах. Крч снова **40**. только впереди - (**1001**).

3) $A < 0$, $B \geq 0$ и $A > B$

Отрицательное число переводится в доп BCD-код и при сложении А и В ответ получается тоже в дополнительном коде, который переводится обратно в прямой.

Пример - 432 + 357:

В BCD со знаком 357 равно 0000 0011 0101 0111. Десятичное дополнение 432 может быть получено, взяв девять дополнений из 432, а затем добавление единицы. Итак, $999 - 432 = 567$ и $567 + 1 = 568$. Если поставить перед 568 в BCD отрицательный знаковый код, можно представить число -432. Итак, -432 в BCD со знаком - это 1001 0101 0110 1000.

Теперь, когда оба числа представлены в BCD со знаком, их можно сложить вместе:

$$\begin{array}{r} 1001\ 0101\ 0110\ 1000\ (9\ 5\ 6\ 8) \\ +\ 0000\ 0011\ 0101\ 0111\ (0\ 3\ 5\ 7) \\ \hline 1001\ 1000\ 1011\ 1111\ (9\ 8\ 11\ 15) \end{array}$$

Поскольку двоично-десятичный формат является формой десятичного представления, некоторые приведенные выше цифровые суммы недействительны. В случае, если существует недопустимая запись (любая цифра BCD больше 1001), добавляется 6 для генерации бита переноса и превращения суммы в действительную запись. Итак, добавление 6 к недопустимым записям приводит к следующему:

$$\begin{array}{r} 1001\ 1000\ 1011\ 1111\ (9\ 8\ 11\ 15) \\ +\ 0000\ 0000\ 0110\ 0110\ (0\ 0\ 6\ 6) \\ \hline 1001\ 1001\ 0010\ 0101\ (9\ 9\ 2\ 5) \end{array}$$

Таким образом, результатом вычитания будет 1001 1001 0010 0101 (-925). Чтобы подтвердить результат, обратите внимание, что первая цифра это 9, что означает минус. Это кажется правильным, поскольку $357 - 432$ должны давать отрицательное число. Остальные полубайты представляют собой двоично-десятичный код, поэтому 1001 0010 0101 равно 925. Десятичное дополнение к 925 составляет $1000 - 925 = 75$, поэтому вычисленный ответ равен -75.

4) $A < 0$, $B \geq 0$ и $A \leq B$

Отрицательное число переводится в доп BCD-код и при сложении А и В ответ получается в прямом коде.

Пример -188 + 521:

Число -188 (1001 0001 1000 1000) в дополнительном коде будет равно 1001 1000 0001 0010.

Число 521 представим как 0000 0101 0010 0001. И сложим:

```
1001 1000 0001 0010
+ 0000 0101 0010 0001
1001 1101 0011 0011
```

Корректируем:

```
1001 1101 0011 0011
+ 0000 0110 0000 0000
1010 0011 0011 0011
```

Еще раз:

```
1010 0011 0011 0011
+ 0110 0000 0000 0000
(1) 0000 0011 0011 0011
```

(1) отбрасывается и мы получаем 0000 0011 0011 0011 (+333) (Т.к число положительное, то дополнительный = прямой код), что и будет ответом.

43. Системы счисления в остаточных классах (СОК)

СОК имеет свой базис, например: базис (2;3;5)

Числа в базисе должны быть взаимно простыми (не иметь общих делителей)

С помощью СОК с базисом (2;3;5) можно закодировать $M = 2 \cdot 3 \cdot 5 = 30$ чисел (от 0 до 29)

Числа для данного базиса кодируются следующим образом:

1-ое число чередуется от 0 до 1 (т.к. 1-ое число базиса = 2)

2-ое число чередуется от 0 до 2 (2-ое число = 3)

3-ое число чередуется от 0 до 4 (3-ое число = 5)

0 = (0;0;0)	1 = (1;1;1)	2 = (0;2;2)	3 = (1;0;3)	4 = (0;1;4)
5 = (1;2;0)	6 = (0;0;1)	7 = (1;1;2)	8 = (0;2;3)	9 = (1;0;4)
10 = (0;1;0)	11 = (1;2;1)	12 = (0;0;2)	13 = (1;1;3)	14 = (0;2;4)
15 = (1;0;0)	16 = (0;1;1)	17 = (1;2;2)	18 = (0;0;3)	19 = (1;1;4)
20 = (0;2;0)	21 = (1;0;1)	22 = (0;1;2)	23 = (1;2;3)	24 = (0;0;4)
25 = (1;1;0)	26 = (0;2;1)	27 = (1;0;2)	28 = (0;1;3)	29 = (1;2;4)

Далее просто пользуемся существующей таблицей

Преимущества СОК:

легко выполняются арифметические операции

Сложим два числа 9 и 14 в базисе (2;3;5):

9 = (1;0;4) 14 = (0;2;4)

Отдельно сложиваем каждый разряд

9	1	0	4
14	0	2	4
14 + 9	1	2	8

Делим по модулю (находим остаток от деления) результат на базис

14 + 9	1	2	8
Базис	2	3	5
Остаток от деления	1	2	3

Сверяем по таблице: (1;2;3) = 23

Аналогично выполняются и остальные действия (но при делении, результат должен быть целочисленным)

Недостатки СОК:

- 1) нельзя сравнить два числа (Что больше: $(1;2;2)$ или $(0;2;0)$?)
- 2) сложно преобразовать СОК-число в обычное и обратно
- 3) сложно поделить два числа если результат дробный
- 4) сложно определить переполнение (в таблице числа 30 и 0, 31 и 1, и т.д. ... имеют одинаковую кодировку)

44. Основные понятия теории кодирования

Кодирование знака числа. Кодирование чисел позволяет заменить операцию арифметического вычитания операцией алгебраического сложения с помощью двоичного сумматора. Для кодирования знака числа используется специальный двоичный разряд, называемый *знаковым*. При этом знак плюс кодируется двоичной цифрой 0, а минус – цифрой 1 (для системы счисления с основанием r – цифрой $r - 1$). Для машинного представления чисел используют три основных вида кодов: прямой, обратный и дополнительный. Общая схема кода числа: код знака. код числа.

прямой, обратный и доп коды см выше, переписывать я не буду (вопросы 15, 16, 17)

45. Контроль передачи информации – контроль на четность, нечетность. Условие обнаружения одиночных ошибок

Если в математическом коде выделен один контрольный разряд ($k = 1$), то к каждому двоичному числу добавляется один избыточный разряд и в него записывается 1 или 0 с таким условием, чтобы сумма цифр в каждом числе была по модулю 2 равна 0 для случая четности или 1 для случая нечетности. Появление ошибки в кодировании обнаружится по нарушению четности (нечетности). При таком кодировании допускается, что может возникнуть только одна ошибка. В самом деле, для случая четности правильной будет только половина возможных комбинаций. Чтобы одна допустимая комбинация превратилась в другую, должно возникнуть по крайней мере два нарушения или четное число нарушений. Пример реализации метода четности представлен в табл. 3.8.

Таблица 3.8 – Пример реализации метода четности

Число	Контрольный разряд	Проверка
10101011	1	0
11001010	0	0
10010001	1	0
11001011	0	1-нарушение

Такое кодирование имеет минимальное кодовое расстояние, равное 2.

Можно представить и несколько видоизмененный способ контроля по методу четности - нечетности. Длинное число разбивается на группы, каждая из которых содержит l разрядов. Контрольные разряды выделяются всем группам по строкам и по столбцам согласно следующей схеме:

a1	a2	a3	a4	a5	k1
a6	a7	a8	a9	a10	k2
a11	a12	a13	a14	a15	k3
a16	a17	a18	a19	a20	k4
a21	a22	a23	a24	a25	k5
k6	k7	k8	k9	k10	

Рисунок 3.10 – Выделение контрольных разрядов по строкам и столбцам

Увеличение избыточности информации приводит к тому, что появляется возможность не только обнаружить ошибку, но и исправить ее. Пусть произошла неисправность в каком-то из разрядов этого числа (представим, что разряд a_{18} изменил состояние, т. е. $a_{18} = 1$). Это приведет к тому, что при проверке на четность сумма $S_i(a_i + k_i)$ по соответствующим строкам и столбцам изменится для значений, которые содержат элемент a_{18} , т. е. это будет четвертая сверху строка и третий слева столбец. Следовательно, нарушение четности по этой строке и столбцу можно зафиксировать, что в конечном счете означает обнаружение не только самой ошибки, но и места, где возникла ошибка. Изменив содержимое отмеченного разряда (в данном случае a_{18}) на противоположное, можно исправить ошибку.

46. Код Хемминга

Допустим, у нас есть сообщение «habr», которое необходимо передать без ошибок. Для этого сначала нужно наше сообщение закодировать при помощи Кода Хемминга. Нам необходимо представить его в бинарном виде.

Символ	ASCII код	Бинарное представление
h	68	01000100
a	61	00111101
b	62	00111110
r	72	01001000

На этом этапе стоит определиться с, так называемой, длиной информационного слова, то есть длиной строки из нулей и единиц, которые мы будем кодировать. Допустим, у нас длина слова будет равна 16. Таким образом, нам необходимо разделить наше исходное сообщение («habr») на блоки по 16 бит, которые мы будем потом кодировать отдельно друг от друга. Так как один символ занимает в памяти 8 бит, то в одно кодируемое слово помещается ровно два ASCII символа. Итак, мы получили две бинарные строки по 16 бит:

h	a	и	b	r
01000100	00111101		00111110	01001000

После этого процесс кодирования распараллеливается, и две части сообщения («ha» и «br») кодируются независимо друг от друга. Рассмотрим, как это делается на примере первой части. Прежде всего, необходимо вставить контрольные биты. Они вставляются в строго определённых местах — это позиции с номерами, равными степеням двойки. В нашем случае (при длине информационного слова в 16 бит) это будут позиции 1, 2, 4, 8, 16. Соответственно, у нас получилось 5 контрольных бит (выделены красным цветом):

Было:

h	a
01000100	00111101

Стало:

h	a
000010000100	001011101

Таким образом, длина всего сообщения увеличилась на 5 бит. До вычисления самих контрольных бит, мы присвоили им значение «0».

Вычисление контрольных бит.

Теперь необходимо вычислить значение каждого контрольного бита. Значение каждого контрольного бита зависит от значений информационных бит (как неожиданно), но не от всех, а только от тех, которые этот контрольный бит контролирует. Для того, чтобы понять, за какие биты отвечает каждый контрольный бит необходимо понять очень простую закономерность: контрольный бит с номером N контролирует все последующие N бит через каждые N бит, начиная с позиции N. Не очень понятно, но по картинке, думаю, станет яснее:

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	
0	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	1	1	1	0	1	
x		x		x		x		x		x		x		x		x		x		x	1
	x	x			x	x			x	x			x	x			x	x			2
			x	x	x	x					x	x	x	x					x	x	4
							x	x	x	x	x	x	x	x							8
															x	x	x	x	x	x	16

Здесь знаком «X» обозначены те биты, которые контролирует контрольный бит, номер которого справа. То есть, к примеру, бит номер 12 контролируется битами с номерами 4 и 8. Ясно, что чтобы узнать какими битами контролируется бит с номером N надо просто разложить N по степеням двойки.

Но как же вычислить значение каждого контрольного бита? Делается это очень просто: берём каждый контрольный бит и смотрим сколько среди контролируемых им битов единиц, получаем некоторое целое число и, если оно чётное, то ставим ноль, в противном случае ставим единицу. Вот и всё! Можно конечно и наоборот, если число чётное, то ставим единицу, в противном случае, ставим 0. Главное, чтобы в «кодирующей» и «декодирующей» частях алгоритм был одинаков. (Мы будем применять первый вариант).

Высчитав контрольные биты для нашего информационного слова получаем следующее:

h	a
100110000100	001011101

и для второй части:

b	r
100101101110	010101000

Декодирование и исправление ошибок.

Теперь, допустим, мы получили закодированное первой частью алгоритма сообщение, но оно пришло к нас с ошибкой. К примеру мы получили такое (11-ый бит передан неправильно):

h	a
100110000110	001011101

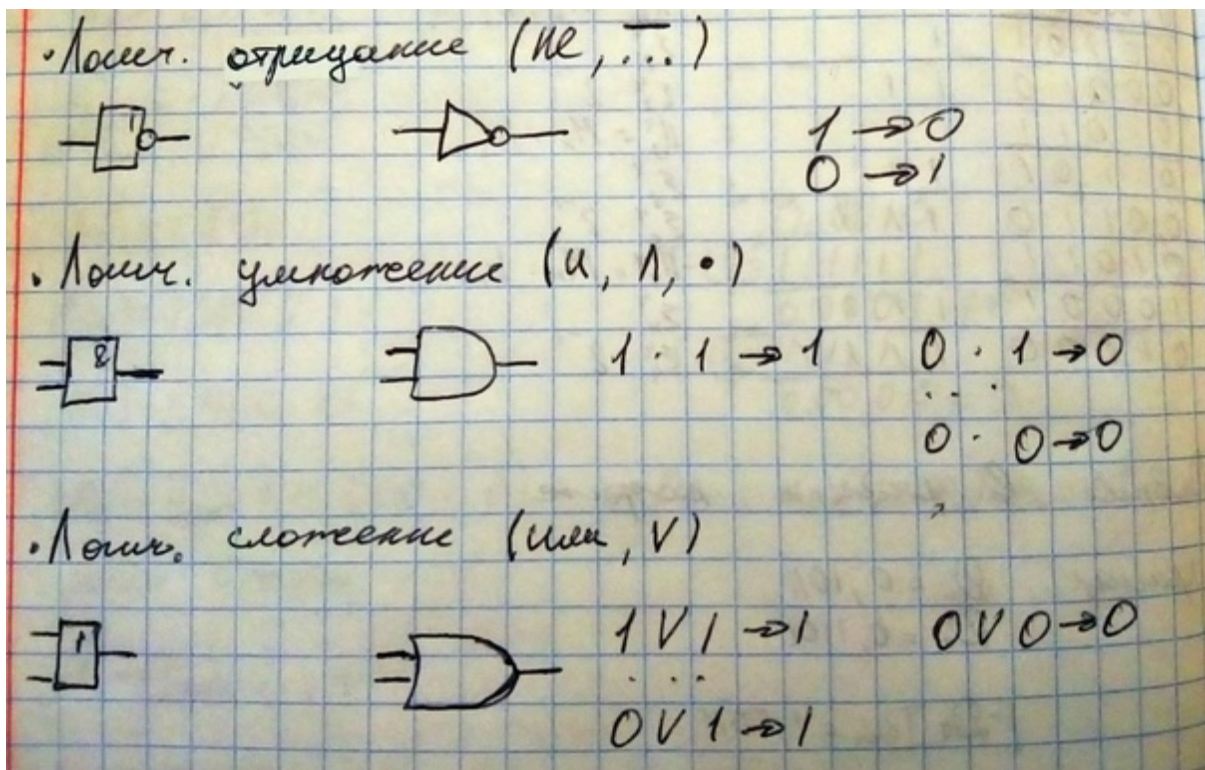
Вся вторая часть алгоритма заключается в том, что необходимо заново вычислить все контрольные биты (так же как и в первой части) и сравнить их с контрольными битами, которые мы получили. Так, посчитав контрольные биты с неправильным 11-ым битом мы получим такую картину:

h	a
010110010110	001011101

Как мы видим, контрольные биты под номерами: 1, 2, 8 не совпадают с такими же контрольными битами, которые мы получили. Теперь просто сложив номера позиций неправильных контрольных бит ($1 + 2 + 8 = 11$) мы получаем позицию ошибочного бита. Теперь просто инвертировав его и отбросив контрольные биты, мы получим исходное сообщение в первоизданном виде! Абсолютно аналогично поступаем со второй частью сообщения.

Ч2. Алгебра логики

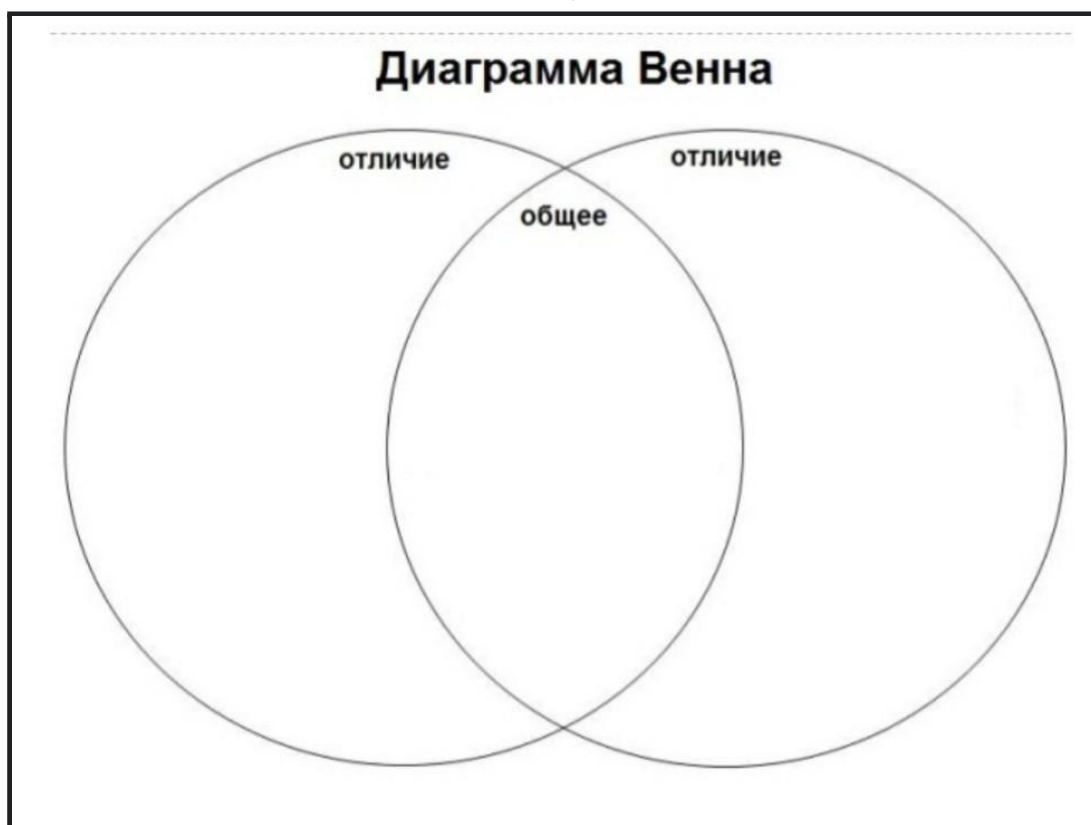
1. Логические элементы И, ИЛИ, НЕ. Функции элементов



2. Диаграммы Венна

Диаграмма Венна — схематичное изображение всех возможных отношений нескольких подмножеств универсального множества.

(эта штука кажется работает как круги Эйлера в логике, только вокруг еще есть прямоугольник, который обычно обозначается буквой U и обозначает он универсальное множество в котором круги являются подмножествами)



Пример:

3. Логический базис. Функциональная полнота логического базиса

Набор простейших функций, с помощью которого можно выразить любые другие, сколь угодно сложные логические функции, называется функционально полным набором, или логическим базисом. Функции **И**, **ИЛИ**, **НЕ** образуют основной логический базис.

Полная система функций называется базисом, если она перестает быть полной при исключении из нее любого элемента.

Функциональная полнота логического базиса — это возможность выразить все возможные значения таблиц истинности с помощью формул из элементов этого множества. Математическая логика обычно использует такой набор операций: конъюнкция ($\wedge$), дизъюнкция ($\vee$), отрицание ($\neg$), импликация ($\rightarrow$) и эквиваленция ($\leftrightarrow$). Это множество операций является

функционально полным. Но оно не является минимальной функционально полной системой, поскольку:

$$A \rightarrow B = \neg A \vee B$$

$$A \leftrightarrow B = (A \rightarrow B) \wedge (B \rightarrow A)$$

Таким образом $\{\neg, \wedge, \vee\}$ также является функционально полной системой. Но $\vee$ также может быть выражено (в соответствии с законом де Моргана) как:

$$A \vee B = \neg(\neg A \wedge \neg B)$$

$\wedge$ также может быть определена через $\vee$ подобным образом.

Также $\vee$ может быть выражена через $\rightarrow$ следующим образом:

$$A \vee B := (A \rightarrow B) \rightarrow B$$

Итак $\{\neg\}$ и одна из $\{\wedge, \vee, \rightarrow\}$ является минимальной функционально полной системой.

4. Классы функций алгебры логики

Функцией алгебры логики $f(x_1, x_2, \dots, x_n)$ от n переменных $x_1, x_2, \dots, x_n$ называется функция, принимающая значения 1 или 0, аргументы которой также принимают значения 1 или 0.

Функции алгебры логики можно классифицировать:

- По количеству аргументов функции, различают нульарные ($n = 0$, булевы константы), унарные ($n = 1$), бинарные ($n = 2$).
- По принадлежности к основным замкнутым классам (линейные, самодвойственные функции);
- По количеству единиц и нулей в таблице истинности различают сбалансированные булевы функции (с одинаковым количеством "0" и "1" в векторе значений) и несбалансированные булевы функции.

5. Функционально полная система функций алгебры логики. Функционально полные наборы

Наибольшее распространение получил набор, в состав которого входят три логические функции:

- f_{10} – инверсия (логическая связь НЕ, логическое отрицание);
- f_1 – конъюнкция (логическая связь И, логическое умножение),
- f_7 – дизъюнкция (логическая связь ИЛИ, логическое сложение).

Этот набор получил название функционально полной системы логических функций (ОФПС). Из теоремы о функциональной полноте следует, что основная функционально полная система логических функций является избыточной, так как условиям теоремы отвечают наборы функций f_{10} и f_1 или f_{10} и f_7 .

Функционально полным называется *такой набор ЛЭ*, на которых (из которых) можно *построить любое логическое устройство* сколь угодно сложно оно ни было бы. Функциональная полнота некоторого набора логических элементов, в свою очередь, определяется полнотой некоторой системы логических функций, которые являются логико-математическими моделями выбранного набора ЛЭ.

6. Определение булевой функции

Булевыми функциями называются двоичные функции $f = f(x_1, x_2, \dots, x_n)$ двоичных (булевых) переменных $(x_1, x_2, \dots, x_n)$ или проще называется любая функция, в которой аргументы и функция могут принимать значение либо 0 либо 1

7. Логический базис. Операции, составляющие булеву алгебру

Основными операциями булевой алгебры являются: отрицание(НЕ), логическое сложение(ИЛИ) и логическое умножение(И).

Базисом называется полная система булевых функций, с помощью которой любая булева функция может быть представлена суперпозицией исходных функций.

Базис может быть избыточным и минимальным.

Минимальный базис – такой базис, когда удаление одной (любой) функции превращает систему булевых функций в неполную. Иначе говоря,

функционально полный базис – это набор операций булевой алгебры (и соответствующих им элементов), позволяющих построить любую булеву функцию.

Функционально полным базисом является базис И, ИЛИ, НЕ. В то же время он функционально избыточен. Удаление из него элемента И или ИЛИ превращает его в минимальный базис.

8. Аксиомы булевой алгебры

- 1) $\overline{\overline{x}} = x$
- 2) $x + x = x$; $x * x = x$
- 3) $x + 0 = x$
- 4) $x + 1 = 1$
- 5) $x * 0 = 0$
- 6) $x * 1 = x$
- 7) $x\overline{x} = 0$
- 8) $x + \overline{x} = 1$

9. Закон поглощения

Законы поглощения показывают, как упрощать логические выражения при повторе операнда:

$$A \vee (A \wedge B) = A;$$

$$A \wedge (A \vee B) = A.$$

10. Закон простого склеивания. Закон обобщенного склеивания

Текст

11. Законы Де-Моргана

Законы де Моргана — логические правила, связывающие пары логических

$$\overline{x_1 \wedge x_2} = \overline{x_1} \vee \overline{x_2};$$

операций при помощи логического отрицания. $\overline{x_1 \vee x_2} = \overline{x_1} \wedge \overline{x_2}$

13. Обобщение законов де Моргана

$$\overline{x_1 x_2 \dots x_n} = \overline{x_1} \vee \overline{x_2} \vee \dots \vee \overline{x_n};$$

$$\overline{x_1 \vee x_2 \vee \dots \vee x_n} = \overline{x_1} \overline{x_2} \dots \overline{x_n}.$$

12. Принцип двойственности

Класс самодвойственных функций. Булевы функции $f_1(x_1, x_2, \dots, x_n)$ и $f_2(x_1, x_2, \dots, x_n)$ называются двойственными друг другу, если выполняется соотношение $f_1(x_1, x_2, \dots, x_n) = \overline{f_2(\overline{x_1}, \overline{x_2}, \dots, \overline{x_n})}$.

К самодвойственным булевым функциям относят такие булевы функции, которые двойственны по отношению к самим себе, т. е. булева функция называется самодвойственной, если на любых двух противоположных наборах $x_1, x_2, \dots, x_n$ и $\overline{x_1}, \overline{x_2}, \dots, \overline{x_n}$ она принимает противоположные значения $f(x_1, x_2, \dots, x_n) = \overline{f(\overline{x_1}, \overline{x_2}, \dots, \overline{x_n})}$.

Любая булева функция, полученная с помощью операции суперпозиции и подстановки из функций одного класса, принадлежит этому же классу.

Базисом называется полная система булевых функций, с помощью которой любая булева функция может быть представлена суперпозицией исходных функций.

13. Определение элементарной конъюнкции.

Пример

Элементарной конъюнкцией называется конъюнкция литералов (переменных или их отрицаний), взятых не более чем по одному разу.

Конъюнкция — логическая операция, по смыслу максимально приближенная к союзу «и». Синонимы: логическое «И», логическое умножение, иногда просто «И».

Конъюнкция может быть *бинарной* операцией (т. е. иметь два операнда), *тернарной* операцией (т. е. иметь три операнда), или *n-арной* операцией (т. е. иметь n операндов).

Наиболее часто встречаются следующие обозначения для операции конъюнкции:

$$a \wedge b, \quad a \&\&b, \quad a \&b, \quad a \cdot b, \quad a \text{ AND } b, \quad \min(a, b)$$

14. Определение полной элементарной конъюнкции. Пример

Элементарная конъюнкция называется полной если содержит символы всех переменных причем некоторые из них могут быть под знаком инверсии.

Пример. Элементарными являются конъюнкции

$1, x_i, x_i x_j, \overline{abc}$, имеющие ранги 0, 1, 2, 3.

Элементарными конъюнкциями не являются:

$\overline{x_1 x_2}, \overline{aba}, \overline{abb}$.

15. Определение элементарной дизъюнкции.

Пример

Элементарной дизъюнкцией называется логическая сумма любого конечного числа различных между собой булевых переменных, взятых со знаком инверсии или без него.

16. Определение полной элементарной дизъюнкции. Пример

Дизъюнкция - это логическое “ИЛИ” (“V”).

Элементарная дизъюнкция называется *полной* относительно переменных $x_1, x_2, \dots, x_n$, если она содержит символы всех переменных (со знаком отрицания или без него).

Пример:

$$F(x_1, x_2, x_3) = x_1 \vee x_2 \vee \overline{x_3}$$

(над x_3 инверсия)

17. Определение ДНФ. Пример

Дизъюнктивная Нормальная Форма (ДНФ) – дизъюнкция элементарных конъюнкций

Пример: $xyz \vee xy\overline{z} \vee qwe \vee a$

где $xyz, xy\overline{z}, qwe, a$ – элементарные конъюнкции,
а между ими стоит дизъюнкция

Второй вид записи: $\vee (xyz; xy\overline{z}; qwe; a)$

18. Определение КНФ. Пример

Конъюнктивная нормальная форма (КНФ) в булевой логике — нормальная форма, в которой булева формула имеет вид конъюнкции дизъюнкций литералов. Конъюнктивная нормальная форма удобна для автоматического доказательства теорем. Любая булева формула может быть приведена к КНФ

Формулы в КНФ:

$$\neg A \wedge (B \vee C),$$
$$(A \vee B) \wedge (\neg B \vee C \vee \neg D) \wedge (D \vee \neg E),$$
$$A \wedge B.$$

Формулы не в КНФ:

$$\neg(B \vee C),$$
$$(A \wedge B) \vee C,$$
$$A \wedge (B \vee (D \wedge E)).$$

Но эти 3 формулы не в КНФ эквивалентны следующим формулам в КНФ:

$$\neg B \wedge \neg C,$$
$$(A \vee C) \wedge (B \vee C),$$
$$A \wedge (B \vee D) \wedge (B \vee E).$$

19. Определение СДНФ (совершенной ДНФ). Пример

Если в математическом коде выделен один контрольный разряд ($k = 1$), то к каждому двоичному числу добавляется один избыточный разряд и в него записывается 1 или 0 с таким условием, чтобы сумма цифр в каждом числе была по модулю 2 равна 0 для случая четности или 1 для случая нечетности. Появление ошибки в кодировании обнаружится по нарушению четности (нечетности). При таком кодировании допускается, что может возникнуть только одна ошибка. В самом деле, для случая четности правильной будет только половина возможных комбинаций. Чтобы одна допустимая комбинация превратилась в другую, должно возникнуть по крайней мере два нарушения или четное число нарушений. Пример реализации метода четности представлен в табл. 3.8.

Таблица 3.8 – Пример реализации метода четности

Число	Контрольный разряд	Проверка
10101011	1	0
11001010	0	0
10010001	1	0
11001011	0	1-нарушение

Такое кодирование имеет минимальное кодовое расстояние, равное 2.

Можно представить и несколько видоизмененный способ контроля по методу четности - нечетности. Длинное число разбивается на группы, каждая из которых содержит l разрядов. Контрольные разряды выделяются всем группам по строкам и по столбцам согласно следующей схеме:

a1	a2	a3	a4	a5	k1
a6	a7	a8	a9	a10	k2
a11	a12	a13	a14	a15	k3
a16	a17	a18	a19	a20	k4
a21	a22	a23	a24	a25	k5
k6	k7	k8	k9	k10	

Рисунок 3.10 – Выделение контрольных разрядов по строкам и столбцам

Увеличение избыточности информации приводит к тому, что появляется возможность не только обнаружить ошибку, но и исправить ее. Пусть произошла неисправность в каком-то из разрядов этого числа (представим, что разряд a_{18} изменил состояние, т. е. $a_{18} = 1$). Это приведет к тому, что при проверке на четность сумма $\sum_i (a_i + k_i)$ по соответствующим строкам и столбцам изменится для значений, которые содержат элемент a_{18} , т. е. это будет четвертая сверху строка и третий слева столбец. Следовательно, нарушение четности по этой

строке и столбцу можно зафиксировать, что в конечном счете означает обнаружение не только самой ошибки, но и места, где возникла ошибка. Изменив содержимое отмеченного разряда (в данном случае a_{18}) на противоположное, можно исправить ошибку.

20. Определение СКНФ (совершенной КНФ).

Пример

Совершенной КНФ (СКНФ) булевой функции от n переменных называется такая КНФ, в которой все дизъюнкции имеют ранг n . СКНФ записывается по таблице истинности согласно правилу: для каждого набора переменных, на котором булева функция принимает нулевое значение, записывается дизъюнкция ранга n и все эти дизъюнкции объединяются знаками конъюнкции; переменная имеет знак инверсии, если на соответствующем наборе имеет единичное значение:

$$f_{\text{СКНФ}} = (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)(x_1 \vee \bar{x}_2 \vee \bar{x}_3)(\bar{x}_1 \vee x_2 \vee x_3)(x_1 \vee x_2 \vee x_3).$$

Элементарные дизъюнкции, образующие СКНФ, называют конституентами (составляющими) нуля (макстерм), так как они соответствуют наборам, при которых функция принимает нулевое значение.

21. Теорема единственности СДНФ

В большинстве источников предлагается принять то, что СДНФ можно представить только одним способом на веру. В иных источниках были найдены два способа доказательства, разберем простейший из них. Это следует из таблицы истинности: каждому набору $F_i(X_1, X_2, \dots, X_n) = 1$ соответствует только одна элементарная конъюнкция. Так:

X1	X2	F(X1,X2)	Элементы дизъюнкции
0	0	1	— — X1*X2
0	1	1	— X1*X2
1	0	0	-----
1	1	1	X1X2

Так, функция в виде СДНФ это дизъюнкция данных элементов, так как в каждом из них присутствуют все логические переменные и каждый элемент существует только один раз, причём по-другому представить их не выйдет. Следовательно, совокупности наборов, на которых функция равна единице, соответствует единственная совокупность элементарных конъюнкций, являющаяся СДНФ заданной функции. Доказательство методом от противного (<http://window.edu.ru/catalog/pdf2txt/528/19528/2740>) сложнее и было найдено только в одном источнике, и потому здесь не приводится.

22. Теорема единственности СКНФ

Теорема единственности СКНФ:

Для любой булевой функции $f(\vec{x})$, не равной тождественной единице, существует единственная СКНФ, ее задающая.

★ **Доказательство:** Поскольку инверсия функции $\neg f(\vec{x})$ равна единице на тех наборах, на которых $f(\vec{x})$ равна нулю, то СДНФ для $\neg f(\vec{x})$ можно записать следующим образом:

$$\neg f(\vec{x}) = \bigvee_{f(x^{\sigma_1}, x^{\sigma_2}, \dots, x^{\sigma_n})=0} (x_1^{\sigma_1} \wedge x_2^{\sigma_2} \wedge \dots \wedge x_n^{\sigma_n}), \text{ где } \sigma_i \text{ обозначает наличие или отсутствие отрицания при } x_i$$

Найдём инверсию левой и правой части выражения:

$$f(\vec{x}) = \neg \left(\bigvee_{f(x^{\sigma_1}, x^{\sigma_2}, \dots, x^{\sigma_n})=0} (x_1^{\sigma_1} \wedge x_2^{\sigma_2} \wedge \dots \wedge x_n^{\sigma_n}) \right)$$

Применяя дважды к полученному в правой части выражению правило де Моргана, получаем:

$$f(\vec{x}) = \bigwedge_{f(x^{\sigma_1}, x^{\sigma_2}, \dots, x^{\sigma_n})=0} (\neg x_1^{\sigma_1} \vee \neg x_2^{\sigma_2} \vee \dots \vee \neg x_n^{\sigma_n})$$

Последнее выражение и является СКНФ. Так как СКНФ получена из СДНФ, которая может быть построена для любой функции, не равной тождественному нулю, то теорема доказана.

23. Алгоритм перехода от таблицы истинности к СДНФ

*Шаг 1. По таблице истинности определить наборы значений переменных, на которых функция равна единице.

Шаг 2. Для каждого набора, на котором функция равна единице, записать полную элементарную конъюнкцию – конституенту единицы, взяв со знаком инверсии ту переменную, которая равна нулю. Без знака инверсии в конституенту единицы записываются переменные, соответствующие

единицам в наборе значений переменных из левой части таблицы истинности.

Шаг 3. Полученные конstituенты единицы соединить знаком дизъюнкции, полученная формула представляет собой СДНФ.

□ Пример

Полная элементарная конъюнкция (конституента единицы)	$x_1 x_2 x_3$	f^1
	0 0 0	0
$\bar{x}_1 \bar{x}_2 x_3$	0 0 1	1
	0 1 0	0
	0 1 1	0
$x_1 \bar{x}_2 \bar{x}_3$	1 0 0	1
$x_1 \bar{x}_2 x_3$	1 0 1	1
$x_1 x_2 \bar{x}_3$	1 1 0	1
$x_1 x_2 x_3$	1 1 1	1

$$f_{\text{СДНФ}}^1(x_1, x_2, x_3) = x_1 x_2 x_3 \vee \bar{x}_1 \bar{x}_2 x_3 \vee x_1 \bar{x}_2 \bar{x}_3 \vee x_1 \bar{x}_2 x_3 \vee x_1 x_2 \bar{x}_3$$

*

24. Алгоритм перехода от таблицы истинности к СКНФ

1. Определить наборы значений, где $f=0$.
2. Для каждого набора записать полную элементарную дизъюнкцию (конституенту нуля, взяв со знаком инверсии ту переменную, которая равна 1, без знака 0 {0 = x ; 1 = $\neg x$ })
3. Соединить все наборы знаком конъюнкции.

Полная элементарная дизъюнкция (конституента нуля)	$x_1 x_2 x_3$	f^1
$x_1 \vee x_2 \vee x_3$	0 0 0	0
	0 0 1	1
$x_1 \vee \bar{x}_2 \vee x_3$	0 1 0	0
$x_1 \vee \bar{x}_2 \vee \bar{x}_3$	0 1 1	0
	1 0 0	1
	1 0 1	1
	1 1 0	1
	1 1 1	1

$$f_{\text{СКНФ}}^1(x_1, x_2, x_3) = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee \bar{x}_3).$$

25. Алгоритм перехода от формулы к ДНФ

1. Выражаем импликацию, участвующую в построении формулы, через дизъюнкцию и отрицание, используя эквивалентность

$$\varphi \rightarrow \psi \equiv \neg\varphi \vee \psi.$$

2. Используя законы де Моргана, переносим все отрицания к переменным и сокращаем двойные отрицания пользуясь законом двойного отрицания.

3. Используя закон дистрибутивности $\varphi \wedge (\psi \vee \chi) \equiv (\varphi \wedge \psi) \vee (\varphi \wedge \chi)$, преобразуем формулу так, чтобы все конъюнкции выполнялись раньше дизъюнкций.

В результате применения пп. 1-3 получается ДНФ, эквивалентная исходной формуле.

26. Алгоритм перехода от ДНФ к СДНФ

Если в какой-то простой конъюнкции недостаёт переменной, например, Z , вставляем в неё выражение:

$$Z \vee \neg Z = 1$$

после чего раскрываем скобки (при этом повторяющиеся дизъюнктивные слагаемые не пишем, так как $Z \vee Z = Z$ по закону идемпотентности). Например:

$$\begin{aligned} X \vee \neg Y \neg Z &= X(Y \vee \neg Y)(Z \vee \neg Z) \vee (X \vee \neg X) \neg Y \neg Z = \\ &= XYZ \vee X \neg Y Z \vee XY \neg Z \vee X \neg Y \neg Z \vee \neg X \neg Y \neg Z = \\ &= XYZ \vee X \neg Y Z \vee XY \neg Z \vee X \neg Y \neg Z \vee \neg X \neg Y \neg Z \end{aligned}$$

Таким образом, из ДНФ получили СДНФ.

27. Алгоритм перехода от КНФ к СКНФ

если в простой дизъюнкции не хватает какой-то переменной, например, z , то добавляем в нее выражение $z \cdot \bar{z} = 0$ (это не меняет самой дизъюнкции), после чего раскрываем скобки с использованием распределительного закона. Пример:

$$(x \vee y)(x \vee \bar{y} \vee \bar{z}) = (x \vee y \vee z \bar{z})(x \vee y \vee \bar{z}) = (x \vee y \vee z)(x \vee y \vee \bar{z})(x \vee \bar{y} \vee \bar{z}).$$

Таким образом, из КНФ получена СКНФ.

28. Алгоритм перехода от ДНФ к КНФ (без построения таблицы истинности)

Алгоритм этого перехода следующий: ставим над ДНФ два отрицания и с помощью правил де Моргана (не трогая верхнее отрицание) приводим отрицание ДНФ снова к ДНФ. При этом приходится раскрывать скобки с использованием правила поглощения (или правила Блейка). Отрицание

(верхнее) полученной ДНФ (снова по правилу де Моргана) сразу дает нам КНФ:

$$xy \vee y\bar{z} = \overline{\overline{xy} \cdot \overline{y\bar{z}}} = \overline{(\bar{x} \vee \bar{y})(\bar{y} \vee z)} = \bar{y} \vee \bar{x}z = y \cdot (\bar{x}z) = y \cdot (x \vee \bar{z}).$$

Заметим, что КНФ можно получить и из первоначального выражения, если вынести y за скобки.

29. Развертывание элементарной конъюнкции в дизъюнкцию полных элементарных конъюнкций

$$*x = xy \vee x(-y)$$

$$x = xyz \vee (xy\bar{z}) \vee (x\bar{y}z) \vee (x\bar{y}\bar{z})$$

Дана:

$$f(a, b, c, d) = a\bar{b}\bar{c} \vee a\bar{c}d \vee \bar{a}b\bar{c} \vee \bar{b}cd$$

Развернем

$$f(a, b, c, d) = a\bar{b}\bar{c}d \vee a\bar{b}\bar{c}\bar{d} \vee a\bar{b}c\bar{d} \vee a\bar{b}cd \vee \bar{a}b\bar{c}d \vee \bar{a}b\bar{c}\bar{d} \vee \bar{a}b\bar{c}d \vee \bar{a}b\bar{c}\bar{d}$$

Получение КНФ. Воспользуемся несколько раз законом дистрибутивности или, проще говоря, раскроем скобки в формуле таким образом, чтобы получить конъюнкцию элементарных дизъюнкций:

$$\begin{aligned} &(\bar{X}_1 \& \bar{X}_3) \vee ((\bar{X}_1 \vee \bar{X}_2) \& (\bar{X}_1 \vee X_2)) = \\ &= ((\bar{X}_1 \& \bar{X}_3) \vee (\bar{X}_1 \vee \bar{X}_2)) \& ((\bar{X}_1 \& \bar{X}_3) \vee (\bar{X}_1 \vee X_2)) = \\ &= (\bar{X}_1 \vee \bar{X}_2 \vee \bar{X}_1) \& (\bar{X}_1 \vee \bar{X}_2 \vee \bar{X}_3) \& (\bar{X}_1 \vee X_2 \vee \bar{X}_1) \& (\bar{X}_1 \vee X_2 \vee \bar{X}_3) \end{aligned}$$

Учитывая, что $A \vee A = A$, перепишем полученную формулу:

$$(\bar{X}_1 \vee \bar{X}_2) \& (\bar{X}_1 \vee \bar{X}_2 \vee \bar{X}_3) \& (\bar{X}_1 \vee X_2) \& (\bar{X}_1 \vee X_2 \vee \bar{X}_3).$$

Эта формула представляет собой КНФ, т.е. конъюнкцию элементарных дизъюнкций. В данном случае имеются четыре элементарные дизъюнкции: $\bar{X}_1 \vee \bar{X}_2$, $\bar{X}_1 \vee \bar{X}_2 \vee \bar{X}_3$, $\bar{X}_1 \vee X_2$, $\bar{X}_1 \vee X_2 \vee \bar{X}_3$. Полученная формула – КНФ, но не СКНФ, так как не во всех элементарных дизъюнкциях содержатся все три переменные. В данном случае в первой и третьей элементарных конъюнкциях не хватает переменной X_3 .

Получение СКНФ. Для этого применяется операция *развертывания*. Она также во многом аналогична операции развертывания, рассмотренной при построении СДНФ. Пусть в некоторой элементарной дизъюнкции (обозначим ее как A) не хватает некоторой переменной B . Элементарная дизъюнкция A заменяется на $A \sqcup 0$ (так как $A \sqcup 0 = A$). Затем, так как $B \& \bar{B} = 0$, записываем формулу $A \sqcup 0$ как $A \vee (B \& \bar{B})$. Далее применяем свойство дистрибутивности: $A \vee (B \& \bar{B}) = (A \vee B) \& (A \vee \bar{B})$. Таким образом, исходная элементарная дизъюнкция A заменяется на две элементарные дизъюнкции, в одной из которых содержится недостающая переменная B , а в другой – ее отрицание. Эта операция выполняется со всеми элементарными дизъюнкциями, пока все они не будут содержать все переменные, имеющиеся в формуле, т.е. пока не будет получена СКНФ.

Применим операцию развертывания к рассматриваемому примеру. Выполним развертывание первой элементарной дизъюнкции, где не хватает переменной X_3 :

$$\begin{aligned}\bar{X}_1 \vee \bar{X}_2 &= \bar{X}_1 \vee \bar{X}_2 \vee 0 = \bar{X}_1 \vee \bar{X}_2 \vee (X_3 \& \bar{X}_3) = \\ &= (\bar{X}_1 \vee \bar{X}_2 \vee X_3) \& (\bar{X}_1 \vee \bar{X}_2 \vee \bar{X}_3)\end{aligned}$$

Аналогично выполним развертывание третьей элементарной конъюнкции:

$$\begin{aligned}\bar{X}_1 \vee X_2 &= \bar{X}_1 \vee X_2 \vee 0 = \bar{X}_1 \vee X_2 \vee (X_3 \& \bar{X}_3) = \\ &= (\bar{X}_1 \vee X_2 \vee X_3) \& (\bar{X}_1 \vee X_2 \vee \bar{X}_3)\end{aligned}$$

Таким образом, все операции развертывания выполнены. Вместо КНФ можно записать следующую формулу:

$$\begin{aligned}(\bar{X}_1 \vee \bar{X}_2 \vee X_3) \& (\bar{X}_1 \vee \bar{X}_2 \vee \bar{X}_3) \& (\bar{X}_1 \vee \bar{X}_2 \vee \bar{X}_3) \& \\ \& (\bar{X}_1 \vee X_2 \vee X_3) \& (\bar{X}_1 \vee X_2 \vee \bar{X}_3) \& (\bar{X}_1 \vee X_2 \vee \bar{X}_3)\end{aligned}$$

Учитывая, что $A \& A = A$, можно переписать эту формулу без повторяющихся элементарных дизъюнций:

$$(\bar{X}_1 \vee \bar{X}_2 \vee X_3) \& (\bar{X}_1 \vee \bar{X}_2 \vee \bar{X}_3) \& (\bar{X}_1 \vee X_2 \vee X_3) \& (\bar{X}_1 \vee X_2 \vee \bar{X}_3)$$

Эта формула – СКНФ для выражения $(X_1 \sqcup X_3) \rightarrow ((X_1 \& X_2) \downarrow (X_1 \& \bar{X}_2))$.

*

30. Матричное (троичной матрицей) представление ДНФ

□ Матричное задание

➤ булева матрица задающая функцию f

➤ ТИ функции заданной матрицей

$$\begin{vmatrix} 001 \\ 010 \\ 011 \\ 101 \\ 111 \end{vmatrix}$$

$x_1 x_2 x_3$	f
0 0 0	0
0 0 1	1
0 1 0	1
0 1 1	1
1 0 0	0
1 0 1	1
1 1 0	0
1 1 1	1

Троичная система: возможные значения 0, 1, -.

31. Числовое представление булевой функции

Более компактной формой записи булевых функций является форма, когда вместо полного перечисления всех конъюнкций (дизъюнкций) используют номера наборов, на которых функция принимает единичное значение. Так, например, форма записи $f(x_1 x_2 x_3) = \vee F(0, 2, 3)$ означает, что функция от трех переменных принимает единичное значение на 0, 2 и 3 наборах таблицы истинности. Такая форма записи называется *числовой*.

32. Определение булева пространства

Булевым пространством называется множество всех наборов – булевых векторов: $M=\{X\}$.

Вместо полного перечисления термов используют номера термов наборов, для которых функция принимает единичные значения.

Например:

$$f(x_1, x_2, x_3) = \vee 1 \vee 3 F(1, 4, 7),$$

Упорядоченную совокупность двоичных переменных $x_1, x_2, \dots, x_n$

можно рассматривать как некоторый переменный вектор $X=(x_1, x_2, \dots, x_n)$ принимающий значение из множества M всех постоянных n -компонентных булевых векторов. Совокупность значений вектора X , на которых булева функция принимает значение «1», обозначим через M^1 , а совокупность значений, на которых функция обращается в «0» - через M^0 . Очевидно, что $M^1 \cap M^0 = \emptyset$ (для полностью определённой булевой функции),

$$f_1(x_1, x_2, \dots, x_n) \in M^1, f_1(x_1, x_2, \dots, x_n) \in M^0.$$

Перечисление этих элементов можно осуществить с помощью булевой матрицы, каждая строка которой задаёт один из элементов множества M^1 .

Например, функция $f_1(x_1, x_2, x_3, x_4)$ принимает значение «1» на трёх кортежах. Булева матрица имеет вид:

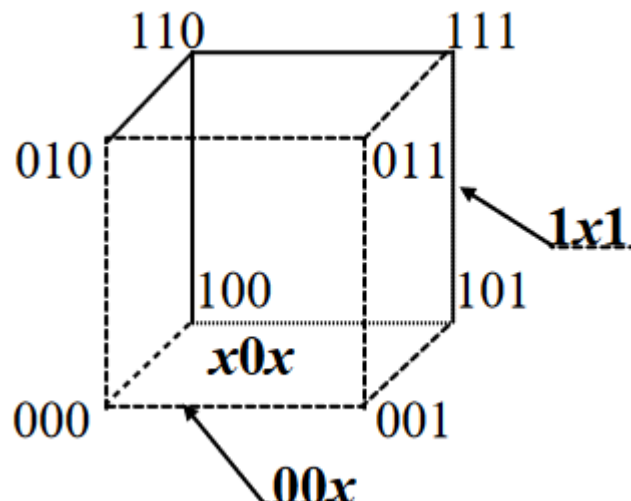
$$\|M^1 \cap X\| = \begin{vmatrix} 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 \end{vmatrix}$$

— кортежи, на которых функция принимает значение 1

33. Представление булева пространства гиперкубом

Пусть у нас есть функция $f(x_1, x_2, x_3)$. Берём 3д пространство, ставим точки 0 и 1 от начала координат по каждой из осей(x, y, z) и строим куб по всем возможным точкам со значениями параметров 0 или 1. Потом смотрим при каких значениях x_1, x_2, x_3 функция принимает значение 1 и выделяем точки с данными координатами. Если 2 соседние точки выделены, то выделяем грань. Если 4 точки на 1 стороне выделены, то мы выделяем сторону, а потом смотрим, что получилось. Точки имеют формулу типа 000, грани $1x1$, стороны куба $x0x$ и это же выражения для минимизации функции (собираем по минимуму эти выражения в ответ). Ниже пример.

	x_1	x_2	x_3	f
0	0	0	0	1
1	0	0	1	1
2	0	1	0	0
3	0	1	1	0
4	1	0	0	1
5	1	0	1	1
6	1	1	0	0
7	1	1	1	1



Ответ: $f(x_1, x_2, x_3) = \text{не } x_2 \text{ или } (x_1 \text{ и } x_3)$

Вообще, гиперкуб может быть квадратом(2 параметра), линией(1 параметр) и т.д. Но рассматривать такие гиперкубы смысла не вижу.

34. Изображение гиперкуба размерности $n=1,2,3,4$

На рис. 24 приведено изображение куба, соответствующего булевой функции от четырех переменных (гиперкуб).

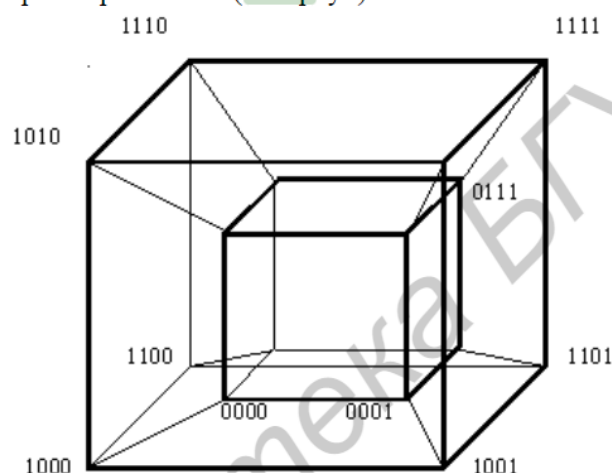


Рис. 24. Геометрическое представление функции четырех переменных

Как следует из рис. 24, геометрическое представление 4-куба уже довольно сложное. Поэтому для функций, зависящих более чем от четырех переменных, предпочтительным является аналитическое представление булевых функций

35. Сформулировать задачу минимизации булевой функции

При проектировании цифровых автоматов широко используются методы минимизации булевых функций, позволяющие получать экономичные схемы. Общая задача минимизации булевых функций может быть сформулирована следующим образом: найти эквивалентное выражение заданной функции алгебры логики в форме, содержащей минимально возможное число букв. Хотя в общем случае под минимизацией может иметься в виду получение выражений с минимальным числом инверсных переменных либо с минимальным числом вхождений какой-либо одной переменной и т. п. Следует отметить, что в общей постановке данная задача пока не решена, однако хорошо исследована в классе дизъюнктивно-конъюнктивных форм.

36. Определение кратчайшей ДНФ

ДНФ функции называется кратчайшей, если она имеет наименьшую длину среди всех эквивалентных ей ДНФ. Кратчайшая ДНФ не обязана быть минимальной, но поиск минимальных проводится среди кратчайших.

37. Определение минимальной ДНФ

Минимальной дизъюнктивной нормальной формой (МДНФ) булевой функции называется ДНФ, содержащая наименьшее число букв (по отношению ко всем другим ДНФ, представляющим заданную булеву функцию).

38. Определение импликанты булевой функции

Булева функция $g(X)$ называется *импликантой* булевой функции $f(X)$, если для любого набора аргументов, на которых $g(X)=1$, $f(X)$ также равна единице.

39. Определение простой импликанты

Простая импликанта – импликанта, к которой нельзя применить операцию склеивания.

40. Определение сокращенной ДНФ

2. Любая булева функция f эквивалентна дизъюнкции всех своих простых импликант. Такая форма представления булевой функции называется *сокращенной ДНФ*.

41. Теорема Квайна

Теорема (теорема Квайна). Если исходя из СДНФ функции произвести все возможные операции неполного склеивания, а затем элементарного поглощения, то в результате получится сокращенная ДНФ, т. е. дизъюнкция всех простых импликант.

42. Метод Квайна нахождения кратчайшей ДНФ

"Метод Квайна основывается на применении двух основных соотношений.

1. Соотношение склеивания

$$Ax \vee A/x = Ax \vee A/x \vee A,$$

где A - любое элементарное произведение.

2. Соотношение поглощения

$$A \sim x \vee A = A, \quad \sim x \in \{x; /x\}.$$

Справедливость обоих соотношений легко проверяется. Суть метода заключается в последовательном выполнении всех возможных склеиваний и затем всех поглощений, что приводит к сокращенной ДНФ. Метод применим к совершенной ДНФ. Из соотношения поглощения следует, что произвольное элементарное произведение поглощается любой его частью.

Для доказательства достаточно показать, что произвольная простая импликанта $p = x_{i1}x_{i2} \dots x_{in}$ может быть получена. В самом деле, применяя к p операцию развертывания (обратную операции склеивания):

$$A = A(x \vee /x) = Ax \vee A/x$$

по всем недостающим переменным $x_{i^{(k+1)}}, \dots, x_{i^n}$ исходной функции f , получаем совокупность S конституент единицы. При склеивании всех конституент из S получим импликанту p . Последнее очевидно, поскольку операция склеивания обратна операции развертывания. Множество S конституент обязательно присутствует в совершенной ДНФ функции f поскольку p - ее импликанта.

Таблица 4.1.1	
$x_4x_3x_2x_1$	f
0000	0
0001	1
0010	0
0011	1
0100	0
0101	1
0110	0
0111	1
1000	0
1001	0
1010	0
1011	0
1100	0
1101	0
1110	1
1111	1

Пример. Пусть имеется булева функция, заданная таблицей истинности (табл. 9.1.1). Ее СДНФ имеет вид

$$f = /x_1/x_2/x_3x_4 \vee /x_1/x_2x_3x_4 \vee /x_1x_2/x_3x_4 \vee /x_1x_2x_3x_4 \vee x_1x_2x_3/x_4 \vee x_1x_2x_3x_4.$$

Для удобства изложения пометим каждую конституенту единицы из СДНФ функции f каким-либо десятичным номером (произвольно). Выполняем склеивания. Конституента 1 склеивается только с конституентой 2 (по переменной x_3) и с конституентой 3 (по переменной x_2), конституента 2 с конституентой 4 и т. д. В результате получаем

$$1 - 2: /x_1/x_2x_4;$$

$$1 - 3: /x_1/x_3x_4;$$

$$2 - 4: /x_1x_3x_4;$$

$$3 - 4: /x_1x_2x_4;$$

$$4 - 6: x_2x_3x_4;$$

$$5 - 6: x_1x_2x_3.$$

Заметим, что результатом склеивания является всегда элементарное произведение, представляющее собой общую часть склеиваемых конституент. Далее производим склеивания получаемых элементарных произведений.

Склеиваются только те произведения, которые содержат одинаковые переменные. Имеет место два случая склеивания:

$$/x_1/x_2x_4 \vee /x_1x_2x_4 = /x_1/x_2x_4 \vee /x_1x_2x_4 \vee /x_1x_4;$$

$$/x_1/x_3x_4 \vee /x_1x_3x_4 = /x_1/x_3x_4 \vee /x_1x_3x_4 \vee /x_1x_4,$$

с появлением одного и того же элементарного произведения $/x_1x_4$. Дальнейшие склеивания невозможны. Произведя поглощения (из полученной ДНФ вычеркиваем все поглощаемые элементарные произведения), получим сокращенную ДНФ:

$$x_2x_3x_4 \vee x_1x_2x_3 \vee /x_1x_4.$$

Переходим ко второму этапу. Для получения минимальной ДНФ необходимо убрать из сокращенной ДНФ все лишние простые импликанты. Это делается с помощью специальной импликантной матрицы Квайна. Строки такой матрицы отмечаются простыми импликантами булевой функции, т. е. членами сокращенной ДНФ, а столбцы - конstituентами единицы, т. е. членами СДНФ булевой функции.

Пример (продолжение). Импликантная матрица имеет вид (табл. 4.1.2).

Таблица 4.1.2						
Простые импликанты	Конституенты единицы					
	$/x_1/x_2/x_3x_4$	$/x_1/x_2x_3x_4$	$/x_1x_2/x_3x_4$	$/x_1x_2x_3x_4$	$x_1x_2x_3/x_4$	$x_1x_2x_3x_4$
$/x_1x_4$	X	X	X	X		
$x_2x_3x_4$				X		X
$x_1x_2x_3$					X	X

Как уже отмечалось, простая импликанта поглощает некоторую конституенту единицы, если является ее собственной частью. Соответствующая клетка импликантной матрицы на пересечении строки (с рассматриваемой простой импликантой) и столбца (с конституентой единицы) отмечается крестиком (табл. 9.1.2). Минимальные ДНФ строятся по импликантной матрице следующим образом:

1. ищутся столбцы импликантной матрицы, имеющие только один крестик. Соответствующие этим крестикам простые импликанты называются базисными и составляют так называемое ядро булевой функции. Ядро обязательно входит в минимальную ДНФ.

2. рассматриваются различные варианты выбора совокупности простых импликант, которые накроют крестиками остальные столбцы импликантной матрицы, и выбираются варианты с минимальным суммарным числом букв в такой совокупности импликант.

Пример (продолжение). Ядром нашей функции являются импликанты $x_1x_2x_3$; $/x_1x_4$. Импликанта $x_2x_3x_4$ - лишняя, так как ядро накрывает все столбцы импликантной матрицы. Поэтому функция имеет единственную тупиковую и минимальную ДНФ:

$$f = x_1x_2x_3 \vee /x_1x_4$$

Следует отметить, что число N крестиков в одной строке всегда является степенью 2. Более того, читатель может легко убедиться в том, что

$$N = 2^{n-k}$$

где k - число букв, содержащихся в простой импликанте.

Заметим также, что используя различные соотношения, можно расширить область применения метода Квайна за пределы совершенной ДНФ."

43. Задача нахождения кратчайшего строчного покрытия булевой матрицы

Стремясь получить кратчайшую ДНФ, мы должны найти кратчайшее строчное покрытие булевой матрицы. Кратчайшее строчное покрытие булевой матрицы используется для решения задач минимализмами булевой функции. Требуется найти такое множество строк данной матрицы, чтобы каждый её столбец имел единицу хотя бы в одной строке из этого множества и при этом данное множество должно быть минимальным. Для нахождения кратчайших покрытий существуют два метода: метод Патрика и метод Закревского.

44. Алгоритм нахождения кратчайшего строчного покрытия булевой матрицы

Алгоритм является итеративным, на каждой итерации повторяются шаг 1-3, пока матрица не станет пустой (не будет содержать столбцов).

Шаг 1

Если в столбце матрицы есть только одна единица, то соответствующая строка включается в решение – строчное покрытие.

Шаг 2

Находится столбец с минимальным числом единиц. В найденном столбце анализируются те строки, где стоят единицы и выбирается строка с максимальным числом единиц. Эта строка включается в решение.

Шаг 3

Из матрицы вычеркиваются столбцы, покрытые выбранными строками на шаге 1, либо одной строкой, выбранной на шаге 1.

Вычеркиваются строки, включенные в покрытие.

Переход на шаг 1, если матрица не является пустой.

Шаг 4

Конец.

Включенные в покрытие строки составляют кратчайшее строчное покрытие булевой матрицы.

45. Метод Квайна-МакКласки нахождения кратчайшей ДНФ

Алгоритм минимизации

Термы, на которых определена функция алгебры логики, записываются в виде их двоичных эквивалентов;

1. Эти эквиваленты разбиваются на группы, в каждую группу входят эквиваленты с равным количеством единиц (нулей);
2. Производится попарное сравнение эквивалентов в соседних группах, с целью формирования термов более низких рангов;
3. Составляется таблица, заголовком строк в которой являются исходные термы, а заголовком столбцов — термы низких рангов;
4. Расставляются метки, отражающие поглощение термов высших рангов (исходных термов) и далее минимизация производится по методу Куайна.

Метод Куайна — способ представления функции в ДНФ или КНФ с минимальным количеством членов и минимальным набором переменных.

Преобразование функции можно разделить на два этапа:

- на первом этапе осуществляется переход от канонической формы к так называемой сокращённой форме;
- на втором этапе — переход от сокращённой формы к минимальной форме.

Первый этап(получение сокращенной формы):

Представим, что заданная функция f представлена в СДНФ. Для осуществления первого этапа преобразование проходит два действия:

1. *Операция склеивания;*
2. *Операция поглощения.*

Второй этап(табличный)(получение минимальной формы):

Как и на первом этапе, в полученном равенстве могут содержаться члены, устранение которых никаким образом не повлияет на конечный результат. Следующий этап минимизации — удаление таких переменных.

46. Метод минимизирующих карт Карно

Можно сформулировать следующие правила минимизации с использованием минимизирующих карт:

1. В карте Вейча (Карно) группы единиц (для получения МДНФ) и группы нулей (для получения МКНФ) необходимо обвести контурами. Внутри контура должны находиться только одноименные значения функции. Этот процесс соответствует операции склеивания или нахождения импликант данной функции.
2. Количество клеток внутри контура должно быть равно степени двойки.
3. При проведении контуров крайние строки карты (верхние и нижние, левые и правые), а также угловые клетки считаются соседними.
4. Каждый контур должен включать максимально возможное количество клеток. В этом случае он будет соответствовать простой импликанте (имплиценте).
5. Все единицы (нули) в карте (даже одиночные) должны быть охвачены контурами. Любая единица (нуль) может входить в контуры произвольное количество раз.
6. Число контуров должно быть минимальным. Множество контуров, покрывающих все единицы (нули) функции образуют тупиковую ДНФ (КНФ). Целью минимизации является нахождение кратчайшей из множества тупиковых форм.
7. В элементарной конъюнкции (дизъюнкции), которая соответствует одному контуру, остаются только те переменные, значение которых не изменяется внутри обведенного контура. Переменные булевой функции входят в элементарную конъюнкцию (для значений функции 1) без инверсии, если их значение на соответствующих координатах равно единице и с инверсией, если равно нулю. Для значений булевой функции, равных нулю, записываются элементарные дизъюнкции, куда переменные входят без инверсии, если их значение на соответствующих координатах равно нулю и с инверсией — если равно единице.

47. Совместная минимизация функций

Совместная минимизация нескольких логических функций (ЛФ) проводится для получения экономичной реализации их комбинационной схемы (КС) за счёт общего использования элементов, реализующих простые импликанты.

48. Определение не полностью определенной (частичной) булевой функции

Функция называется не полностью “частично” определенной, если для некоторых наборов $x_1, x_2, \dots$ значения функции не заданы.

Пример:

X1	X2	X3	Значение функции
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	dc
1	0	0	dc
1	0	1	1
1	1	0	0
1	1	1	1

Где dc - неопределенное значение (1 или 0).

*Булевы функции являются частным случаем не полностью определенных функций. Также не полностью определенные булевы функции могут быть доопределены (замена dc на 1 или 0).

49. Определение отношения реализации частичных булевых функций

0	-	0	-
1	-	0	1
1	1	-	1
-	0	0	1

При минимизации частичной булевой функции неопределенные значения функции (черточки) доопределяются до 0 или 1, при этом выбираются наименьшее количество блоков с максимальным количеством значений.

4.9.7. Алгоритм извлечения (метод Рота)

Метод Рота [3, 14] предназначен для минимизации булевой функции, заданной в форме произвольного кубического покрытия, что позволяет упростить процесс подготовки выражения для минимизации. Достоинство алгоритма Рота – полная формализация действий на всех этапах минимизации функции.

Специальные логические операции алгоритма Рота: *, ∩,

Реализация алгоритма извлечения осуществляется на основе специальных логических операций, которые позволяют полностью формализовать процесс получения минимальной формы.

Операция умножения кубов (*). Операция умножения кубов $a = a_1a_2...a_n$ и $b = b_1b_2...b_n$ обозначается как $c = a*b$ и служит для образования r -куба, противоположные $(r - 1)$ грани которого содержатся в кубах a и b . Предварительные координаты куба $c = c_1c_2...c_n$ определяются в соответствии с таблицей, приведенной ниже.

		b_i			
		*	0	1	x
a_i	0	0	y	0	
	1	y	1	1	
	x	0	1	x	

Окончательно координаты куба c формируются:

$$a*b = \begin{cases} m(a_1*b_1) m(a_2*b_2) \dots m(a_n*b_n), & \text{если } a_i*b_i = y \text{ только для одного } i, \\ \emptyset, & \text{если } a_i*b_i = y \text{ для } i \geq 2, \end{cases}$$

где $m(a_i*b_i)$ – окончательная i -я координата куба c ; $m(0) = 0$; $m(1) = 1$; $m(y) = x$; y – условное обозначение того, что координаты a_i и b_i противоположны.

Эта операция соответствует операции склеивания: образуется новый r -куб, если кодовое расстояние двух исходных кубов равно единице.

Рассмотрим некоторые примеры, графическая интерпретация которых приведена на рис. 37.

$$\begin{array}{r} 1. \quad 011 \\ \quad *001 \\ \hline \end{array}$$

$0y1 \Rightarrow 0x1$ – ребро покрывает две вершины

$$\begin{array}{r} 2. \quad 11x \\ \quad *01x \\ \hline \end{array}$$

$y1x \Rightarrow x1x$ – грань

$$\begin{array}{r} 3. \quad 0x1 \\ \quad *1x0 \\ \hline \end{array}$$

$yxu \Rightarrow \emptyset$ – новый куб не порождается
(координаты имеют значение y)

$$\begin{array}{r} 4. \quad x1x \\ \quad *011 \\ \hline 011 \end{array}$$

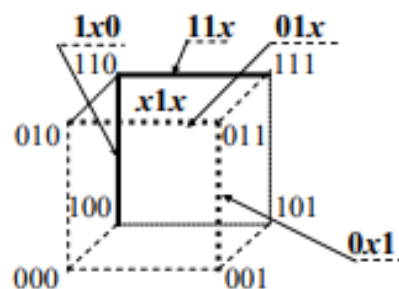


Рис. 37. Графическая интерпретация операции умножения кубов (*)

Операция пересечения кубов ($\cap$). Операция пересечения кубов $a = a_1a_2...a_n$ и $b = b_1b_2...b_n$ обозначается как $c = a \cap b$ и служит для выделения куба $c = c_1c_2...c_n$, являющегося общей частью кубов a и b . Предварительные координаты куба $c = c_1c_2...c_n$ определяются согласно следующей таблице.

		b_i			
		$\cap$	0	1	x
a_i	0	0	$\emptyset$	0	
	1	$\emptyset$	1	1	
	x	0	1	x	

Окончательно координаты куба c формируются

$$a \cap b = \begin{cases} m(a_1 \cap b_1) m(a_2 \cap b_2) \dots m(a_n \cap b_n), \\ \emptyset, \text{ если существует такое } i, \text{ для которого } a_i \cap b_i = \emptyset, \end{cases}$$

где $m(a_i * b_i)$ – окончательная i -я координата куба c ; $m(0) = 0$; $m(1) = 1$; $m(x) = x$.

Рассмотрим примеры и их графическую интерпретацию (рис. 38).

1. $\begin{matrix} 100 \\ \cap \\ 001 \end{matrix}$
 $\emptyset 0 \emptyset \Rightarrow \emptyset$ – общей части у кубов нет

2. $\begin{matrix} 1x0 \\ \cap \\ 10x \end{matrix}$
 $100 \Rightarrow 100$ – общая часть кубов (ребер) – вершина

3. $\begin{matrix} x1x \\ \cap \\ 0xx \\ 01x \end{matrix}$
 $01x \Rightarrow 01x$ – общая часть кубов (граней) – ребро

4. $\begin{matrix} 0xx \\ \cap \\ 0x0 \end{matrix}$
 $0x0$ (совпадает с операцией умножения кубов (*))

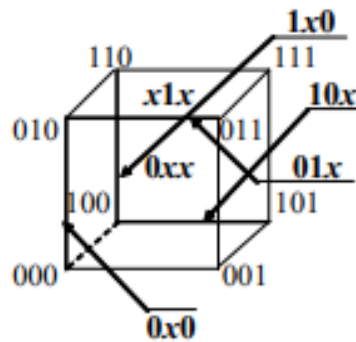


Рис. 38. Графическая интерпретация операции пересечения кубов ($\cap$)

Операция вычитания кубов ($\#$). Операция вычитания из куба $a = a_1a_2...a_n$ куба $b = b_1b_2...b_n$ обозначается как $c = a \# b$ и служит для удаления из куба a общей части кубов a и b . Предварительные координаты куба $c = c_1c_2...c_n$ определяются согласно следующей таблице.

		b_i			
		#	0	1	x
a_i	0	z	y	z	
	1	y	z	z	
	x	1	0	z	

Окончательно координаты куба c формируются

$$c = a \# b = \begin{cases} \emptyset, & \text{если для любого } i \quad a_i \# b_i = z, \\ a, & \text{если существует } i, \text{ такое, что } a_i \# b_i = y, \\ \bigcup_{i=1}^n a_1a_2...a_{i-1}\alpha_i a_{i+1}...a_n, & \text{если } \alpha_i \text{ равно 0 или 1 для одного или нескольких } i, \end{cases}$$

где z означает, что координаты совпадают, а y , как для операции умножения кубов ($*$), означает, что координаты a_i и b_i противоположны.

По этим α_i -координатам производится объединение ($\cup$) кубов, получаемых в результате замены в кубе a символа x на соответствующее значение (0,1) координаты α_i . Рассмотрим примеры выполнения операции вычитания кубов ($\#$) и их графическую интерпретацию (рис. 39).

$$1. \begin{array}{l} x1x \\ \# \\ x11 \\ \hline zz0 \Rightarrow x10 - \text{ребро} \end{array}$$

$$3. \begin{array}{l} xx1 \\ \# \\ x10 \\ \hline z0y \Rightarrow xx1 - \text{грань} \end{array}$$

$$2. \begin{array}{l} x1x \\ \# \\ 110 \\ \hline 0z1 \Rightarrow \{01x, x11\} - \text{два ребра} \end{array}$$

$$4. \begin{array}{l} x11 \\ \# \\ xx1 \\ \hline zzz \Rightarrow \emptyset - \text{пусто} \end{array}$$

Diagram of a 3D coordinate system with axes labeled x_{10} , 010 , and $01x$. The vertices are labeled with binary coordinates: 000 , 001 , 010 , 011 , 100 , 101 , 110 , and 111 . The edges are labeled with binary values: $11x$, $x1x$, and $x11$.

Далее рассмотрим метод Рота на примере минимизации булевой функции, заданной покрытием C_0 (рис. 40).

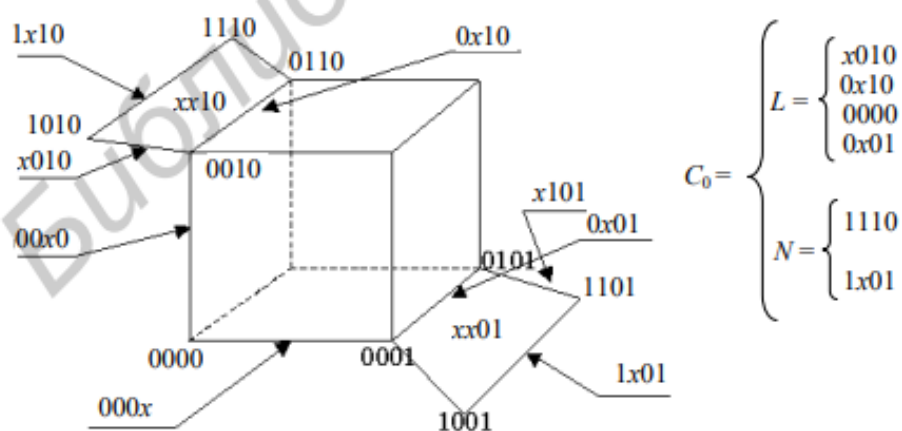


Рис. 40. Графическое задание исходного покрытия

Исходное покрытие C_0 задано объединением множеств кубов L и N . Кубы комплекса N – это наборы, на которых функция не определена и которые включаются в покрытие ради возможного дополнительного упрощения комплекса L . Минимальное покрытие $C_{\min}$ комплексов L и N называется K -покрытием L .

Общий метод построения минимального K -покрытия называется алгоритмом извлечения и состоит в следующем:

- нахождении множества Z простых импликант комплекса K ;
- выделении L -экстремалей на множестве Z ;
- применении алгоритма ветвления при отсутствии L -экстремалей;
- нахождении абсолютно минимального покрытия из некоторого множества избыточных покрытий.

1. Нахождение множества простых импликант

Преобразование исходного покрытия C_0 комплекса K в множество простых импликант Z осуществляется с помощью операции умножения кубов. В результате первого шага ($C_0 * C_0$) (табл. 17) предусматривается выявление как новых кубов C_y (первой и более высокой размерности), так и кубов, которые не образуют новых кубов (включаются в множество Z_0).

Таблица 17

$C_0 * C_0$	x010	0x10	0000	0x01	1110	1x01
x010	–					
0x10	0010	–				
0000	00y0	00y0	–			
0x01	$\emptyset$	$\emptyset$	000y	–		
1110	1y10	y110	$\emptyset$	$\emptyset$	–	
1x01	$\emptyset$	$\emptyset$	$\emptyset$	yx01	$\emptyset$	–

A_1 00x0 x110 000x xx01
 1x10

Из полученных новых кубов образуется множество A_1 . Также формируется множество $B_1 = C_0 - Z_0$. Для следующего шага получения множества Z формируется множество $C_1 = A_1 \cup B_1$. Для уменьшения мощности множества кубов C_1 выполним операцию поглощения (удаления) кубов, образующих множество C_1 , кубами из множества A_1 ($A_1 \subseteq C_1$). Для рассматриваемого примера получим

$$A_1 = \begin{Bmatrix} 00x0 \\ 1x10 \\ x110 \\ 000x \\ xx01 \end{Bmatrix}, \quad Z_0 = \emptyset.$$

Далее сформируем новое множество кубов $C_1 = A_1 \cup B_1$.

$$C_1 = \left\{ \begin{array}{l} 00x0 \\ 1x10 \\ 000x \\ x110 \\ xx01 \\ x010 \\ 0x10 \\ 0000 \\ 0x01 \\ 1110 \\ 1x01 \end{array} \right\} \quad \begin{array}{l} \text{(до выполнения} \\ \text{операции погло-} \\ \text{щения кубами} \\ \text{множества } A \\ \text{кубов множест-} \\ \text{ва } B) \end{array} \quad C_1 = \left\{ \begin{array}{l} 00x0 \\ 1x10 \\ 000x \\ x110 \\ xx01 \\ x010 \\ 0x10 \end{array} \right\} \quad \begin{array}{l} \text{(после выполне-} \\ \text{ния операции} \\ \text{поглощения)} \end{array}$$

$$C_1 * C_1 = (A_1 \cup B_1) * (A_1 \cup B_1) = (A_1 * A_1) \cup (A_1 * B_1) \cup (B_1 * A_1) \cup (B_1 * B_1) = \\ = (A_1 * A_1) \cup (A_1 * B_1).$$

Среди кубов C_0 , возможно, находятся такие кубы, которые с кубами множества A_1 могут дать новые кубы или оказаться простыми импликантами после второго шага ($C_1 * C_1$). При формировании таблицы для выполнения операции $C_1 * C_1$ (табл. 18) следует учесть, что $B_1 * B_1$ уже выполнялось на шаге $C_0 * C_0$. Следовательно

Таблица 18

$C_1 * C_1$	00x0	1x10	000x	x110	xx01
00x0	—				
1x10	y010	—			
000x	0000	∅	—		
x110	0y10	1110	∅	—	
xx01	000y	∅	0001	∅	—
x010	0010	1010	00y0	xy10	∅
0x10	0010	yx10	00y0	0110	∅
A_2	∅	xx10	∅	xx10	∅

В результате выполнения умножения $C_1 * C_1$ получим:

$$A_2 = \{xx10\},$$

$$Z_1 = \left\{ \begin{array}{l} 00x0 \\ 000x \end{array} \right\}.$$

Необходимо отметить, что куб $xx01$ не дал нового куба. Но это куб второй размерности, поэтому новые кубы он может дать на третьем шаге ($C_2 * C_2$) (табл. 19). Таким образом, его не следует включать в число кубов, образующих множество Z_1 .

$$B_2 = \begin{Bmatrix} 1x10 \\ x110 \\ xx01 \\ x010 \\ 0x10 \end{Bmatrix}, \quad C_2 = A_2 \cup B_2 = \begin{Bmatrix} xx10 \\ 1x10 \\ x110 \\ x010 \\ 0x10 \\ xx01 \end{Bmatrix} = \begin{Bmatrix} xx10 \\ xx01 \end{Bmatrix}.$$

Таблица 19

$C_2 * C_2$	$xx10$
$xx10$	—
$xx01$	$\emptyset$
A_3	$\emptyset$

Из табл. 19 следует, что $A_3 = \emptyset$. Таким образом, новых кубов при выполнении операции $C_2 * C_2$ не было получено.

$$Z_2 = \begin{Bmatrix} xx10 \\ xx01 \end{Bmatrix}, \quad B_3 = C_2 \setminus Z_2 = \emptyset, \quad C_3 = A_3 \cup B_3 = \emptyset.$$

На этом процесс выявления простых импликант окончен. Таким образом, сформировано множество простых импликант

$$Z = \bigcup_{i=0}^2 Z_i = Z_0 \cup Z_1 \cup Z_2 = \begin{Bmatrix} 00x0 \\ 000x \\ xx01 \\ xx10 \end{Bmatrix}.$$

Необходимо выяснить, не содержатся ли в этом множестве «лишние» простые импликанты.

2. Определение L -экстремалей

Множество Z может быть избыточным. Прежде всего необходимо выявить обязательные простые импликанты, называемые в алгоритме извлечения L -экстремальными. L -экстремаль – это куб, который (и только он) покрывает некоторую вершину из множества L , не покрываемую никаким другим кубом из множества Z .

Для определения L -экстремалей воспользуемся операциями вычитания ($\#$) (табл. 20) и пересечения ($\cap$) кубов (табл. 21). В табл. 20 $z \subseteq Z$ – некоторая простая импликанта, из которой вычитаются остальные $Z - z$.

Таблица 20

$z\#(Z - z)$	00x0	000x	xx01	xx10
1	2	3	4	5
00x0	—	zzz1 0001	1lzy xx01	1lzz 1x10 x110

Окончание табл. 20

1	2	3	4	5
000x	zz1z 0010	—	11zz 1x01 x101	ylyz 1x10 lyyz x110
xx01	zzyy 0010	zzzz ∅	—	zzyy 1x10 zzyy x110
xx10	zzzz ∅	∅	zzyy 1x01 zzyy x101	—
Остаток	∅	∅	1x01, x101	1x10, x110

Таким образом, из таблицы получено множество L -экстремалей:

$$E = \begin{Bmatrix} xx01 \\ xx10 \end{Bmatrix}.$$

Если результат вычисления $\emptyset$ хотя бы в одном любом случае, то это значит, что среди простых импликант есть кубы, которые покрывают *уменьшаемый*, а следовательно, этот уменьшаемый не может быть L -экстремалью.

Если же полученный результат не $\emptyset$, то в противоположность предыдущему утверждению *уменьшаемый* куб оказывается кубом большей размерности по отношению к другим простым импликантам.

Что касается простых импликант, «удаленных» от *уменьшаемой*, то они с ней дают координаты u и, таким образом, остается *уменьшаемый* куб при вычитании этих «удаленных» кубов.

После выявления L -экстремалей следует выяснить, не являются ли некоторые из них простыми импликантами, остатки которых покрывают только некоторое подмножество кубов комплекса N , которое нет необходимости покрывать, вводя в минимальное покрытие соответствующие наборы. Для этого необходимо выполнить операцию пересечения остатков, полученных при выполнении операции $z \# (Z - z)$ с кубами из комплекса L . Во множестве E необходимо оставить только те кубы, остатки от которых пересекаются с кубами из комплекса L .

Таблица 21

$Z \# (Z - z) \cap L$	1x01	x101	1x10	x110
x010	∅	∅	1010	∅
0x10	∅	∅	1010	∅
0000	∅	∅	∅	0110
0x01	∅	1101	∅	∅

Из табл. 21 видно, что куб 1x01 не пересекается с кубами комплекса L . Однако куб x101 имеет с кубом 0x01 (из комплекса L) общую вершину 0101.

Оба куба (1x01, x101) входят в куб более высокой размерности xx01 (L -экстремаль). Таким образом, куб 1x01, образованный на комплексе N , позволил уменьшить цену схемы. Выясним далее, какие из вершин комплекса L не покрываются L -экстремальными. Для этого из каждого куба комплекса L вычтем (#) элементы множества E (табл. 22). В результате вычитания получим $L_1 = L \# E$.

Таблица 22

$L \# E$	x010	0x10	0000	0x01
xx01	zzyy x010	zzyy 0x10	zzzy 0000	zzzz $\emptyset$
xx10	zzzz $\emptyset$	zzzz $\emptyset$	zzyz 0000	$\emptyset$
	$\emptyset$	$\emptyset$	0000	$\emptyset$

Из табл. 22 видно, что $L_1 = \{0000\}$. Однако не покрытые L -экстремальными кубы должны быть покрыты другими импликантами из множества

$$\hat{Z} = Z \setminus E = \left\{ \begin{array}{l} 00x0 \\ 000x \\ xx01 \\ xx10 \end{array} \right\} \setminus \left\{ \begin{array}{l} xx01 \\ xx10 \end{array} \right\} = \left\{ \begin{array}{l} 00x0 \\ 000x \end{array} \right\}.$$

Теперь из полученного множества $\hat{Z}$ надо выбрать минимальное число кубов с минимальной ценой (максимальной размерностью), чтобы покрыть непокрытые L -экстремальными элементы комплекса L . Выбор так называемого не максимального куба осуществляется с помощью операции частичного упорядочивания кубов (табл. 23). Введем обозначения кубов a и b (кубов может быть и больше):

$$\hat{Z} = \left\{ \begin{array}{l} 00x0 \\ 000x \end{array} \right\} \begin{array}{l} \text{— куб } a, \\ \text{— куб } b. \end{array}$$

Куб a будет не максимален по отношению к кубу b , если выполняются одновременно два условия:

- 1) $C^a \geq C^b$, где C^a — цена куба a ;
- 2) $a \cap L_1 \subseteq b \cap L_1$, куб b покрывает не меньше кубов, чем куб a .

Таблица 23

	$\cap$	0000
a	00x0	0000
b	000x	0000

В результате получаем, что $C^a = C^b$ (пересечение обоих кубов дало в результате 0-куб). Следовательно, кубы a и b равноценны и для покрытия вершины 0000 можно выбрать любой из них в качестве экстремали второго порядка:

$$E_2' = \{000x\} \text{ или } E_2'' = \{00x0\}.$$

Следовательно, могут быть получены две тупиковые формы:

$$f_{\text{МДНФ}}^1 = C_{\min} = E \cup E_2' = \begin{Bmatrix} xx10 \\ xx01 \\ 000x \end{Bmatrix} - \text{первая тупиковая форма,}$$

$$f_{\text{МДНФ}}^2 = C_{\min} = E \cup E_2'' = \begin{Bmatrix} xx10 \\ xx01 \\ 00x0 \end{Bmatrix} - \text{вторая тупиковая форма.}$$

Аналитически полученные тупиковые формы могут быть записаны следующим образом:

$$f_{\text{МДНФ}}^1 = x_3 \bar{x}_4 \vee \bar{x}_3 x_4 \vee \bar{x}_1 \bar{x}_2 \bar{x}_3,$$

$$f_{\text{МДНФ}}^2 = x_3 \bar{x}_4 \vee \bar{x}_3 x_4 \vee \bar{x}_1 \bar{x}_2 \bar{x}_4.$$

Схемная реализация первой тупиковой формы булевой функции приведена на рис. 41.

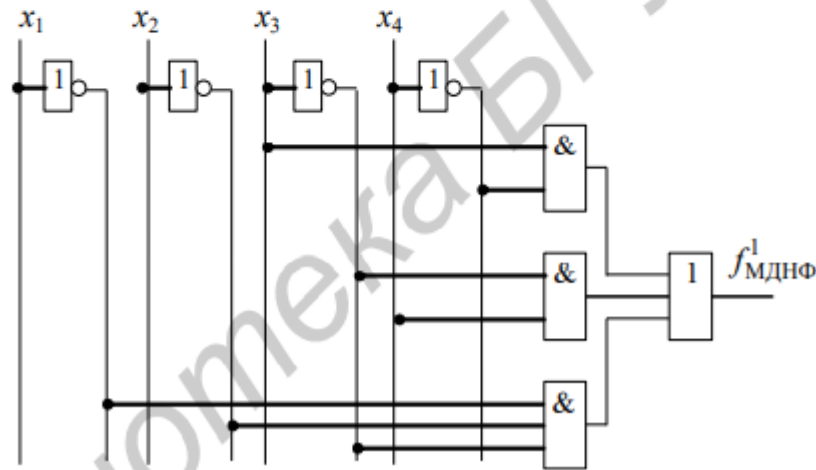


Рис. 41. Схемная реализация $f_{\text{МДНФ}}^1$